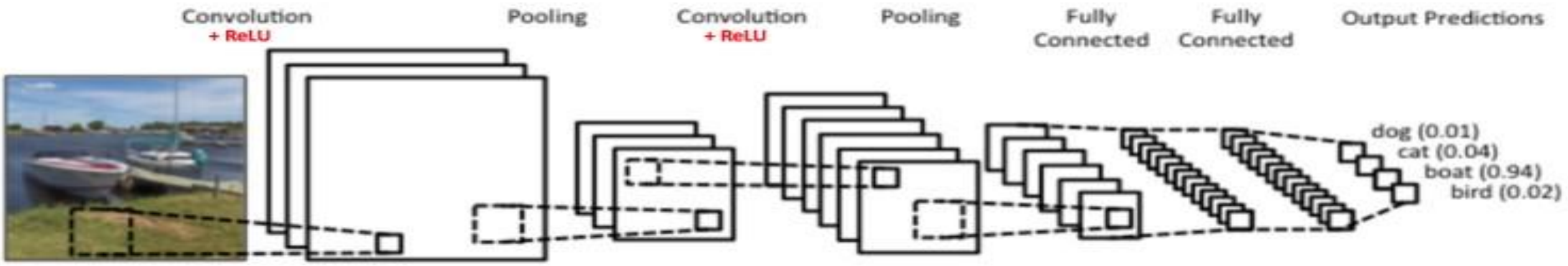
Redes de Convolução: Arquiteturas

Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

INSTITUTO FEDERAL
Goiás

Recapitulando: Rede de Convolução



Recapitulando: Rede de Convolução

- CNNs são inspiradas no conceito de campo receptivo local.

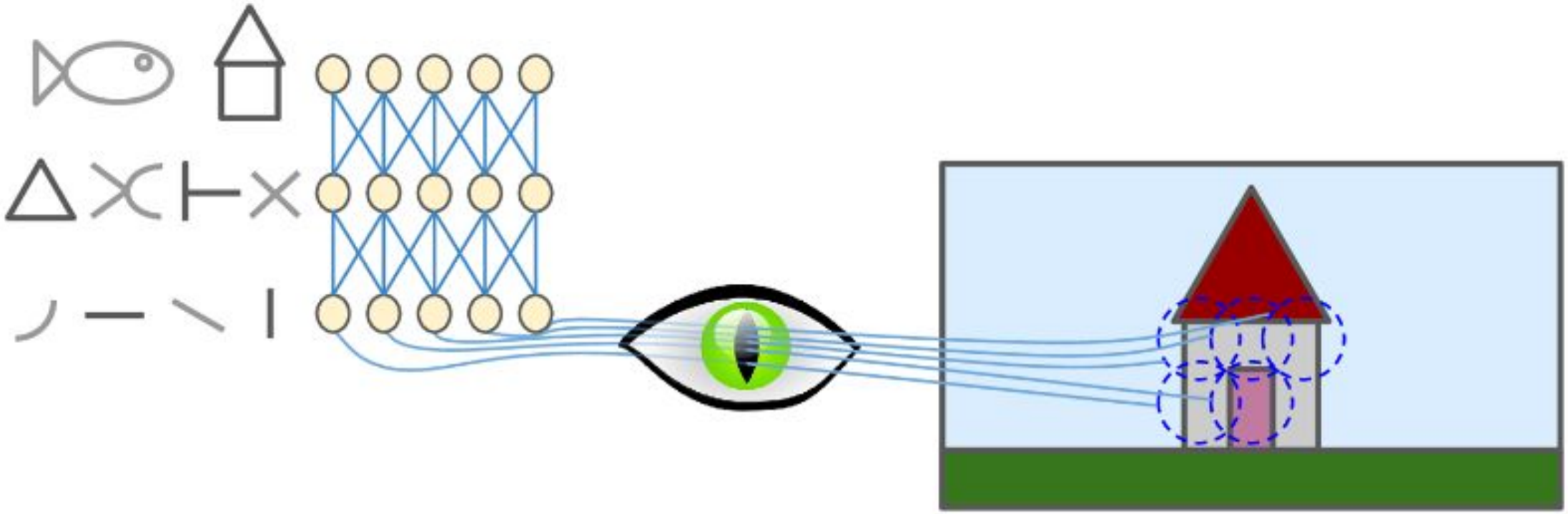


Figure 3-1. Local receptive fields in the visual cortex

Recapitulando: Rede de Convolução - Filtros

- Se utilizam de filtros (kernels) para obter mapas de ativação que evidenciam padrões do filtro na imagem.

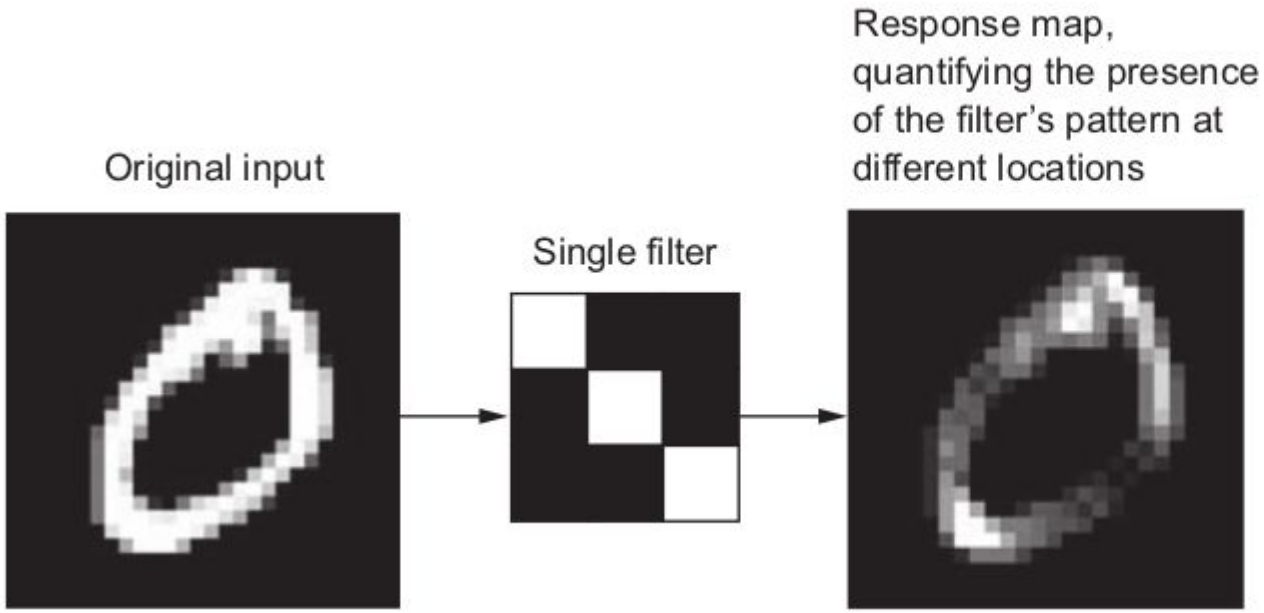


Figure 5.3 The concept of a *response map*: a 2D map of the presence of a pattern at different locations in an input

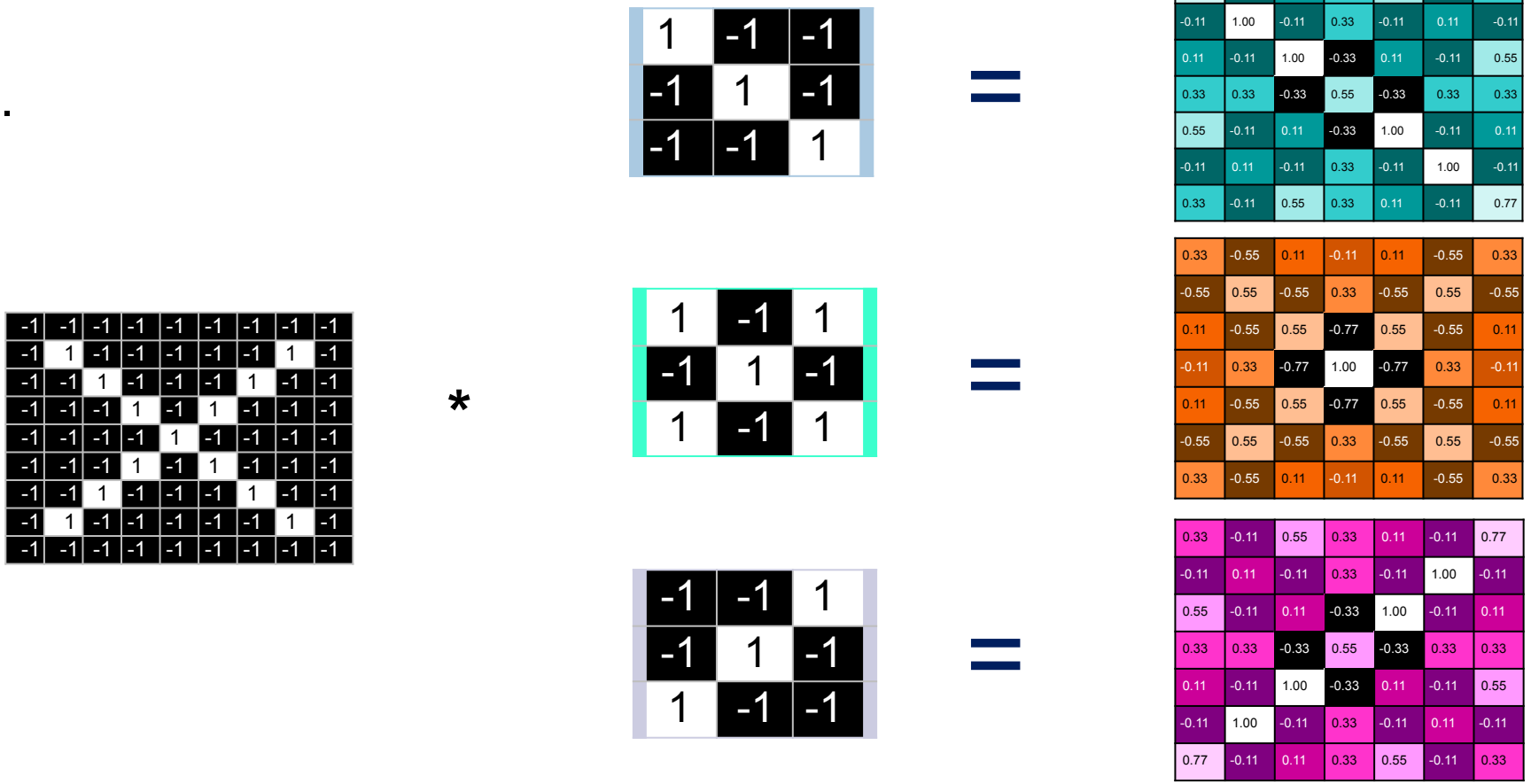
Recapitulando: Rede de Convolução - Filtros

Mapas de ativação:

- Respostas aos filtros.

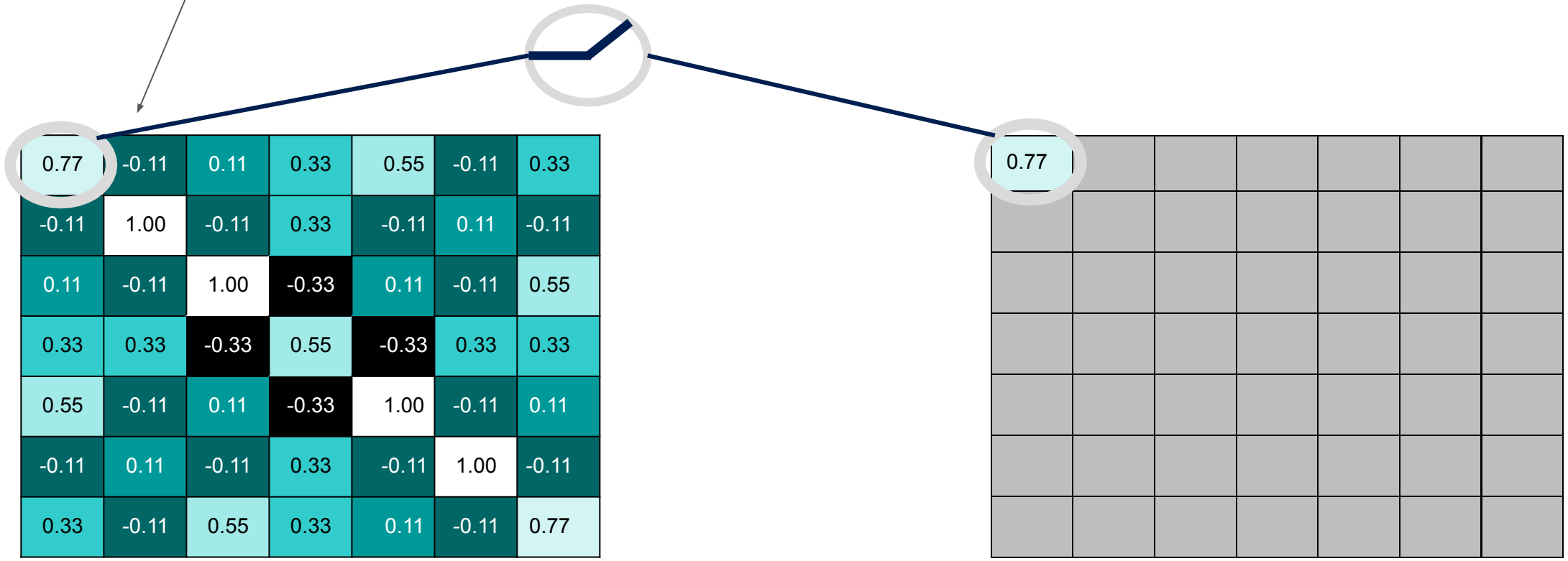
Filtros:

- Kernels.

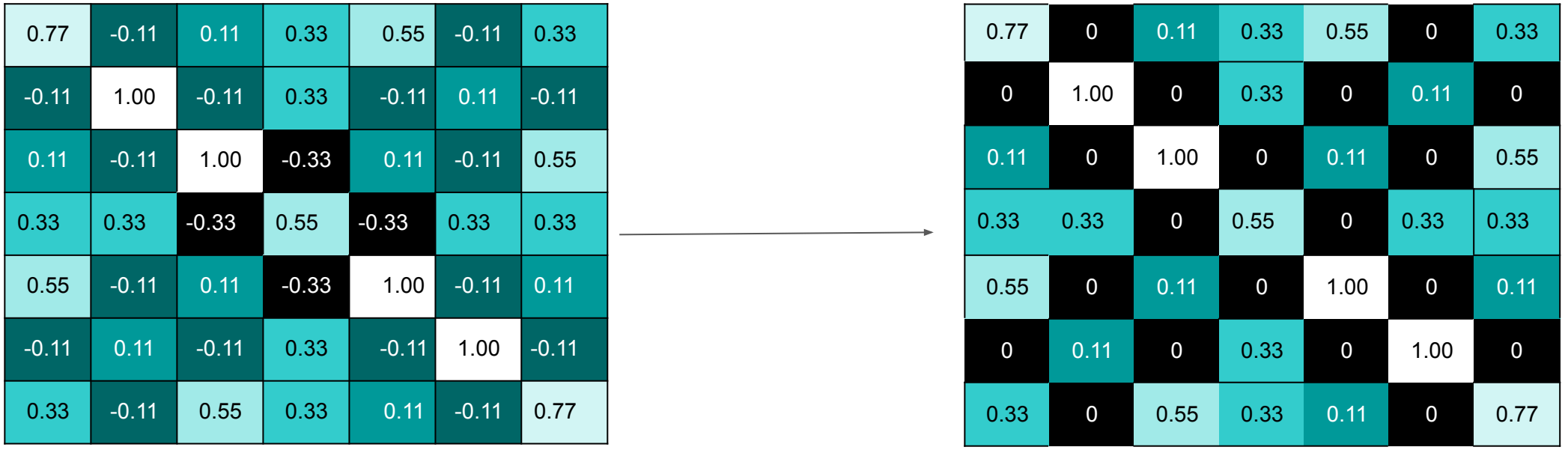


Recapitulando: Redes de convolução - Função de ativação

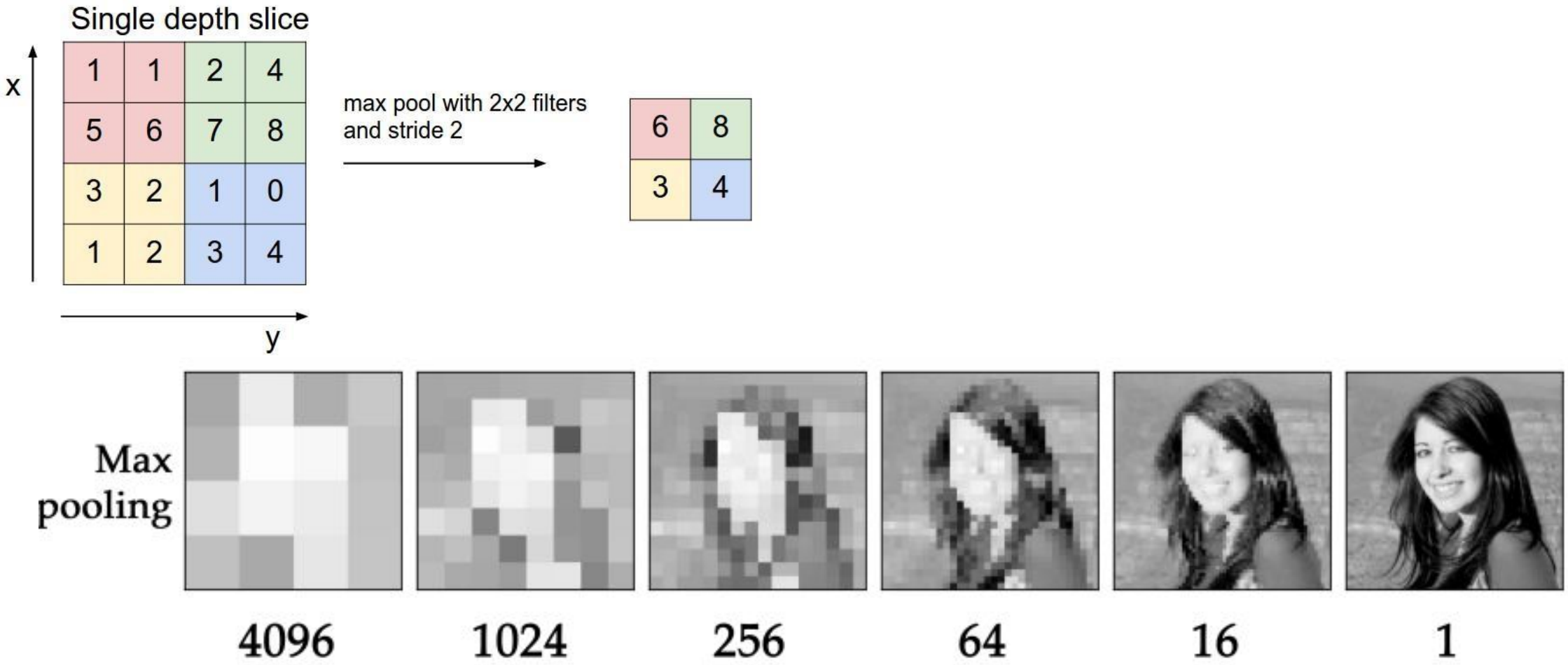
- Utilizo o mapa de ativação (gerado a partir de uma entrada e um filtro treinado) para aplicar uma função de ativação (relu)



Recapitulando: Redes de convolução - Função de ativação



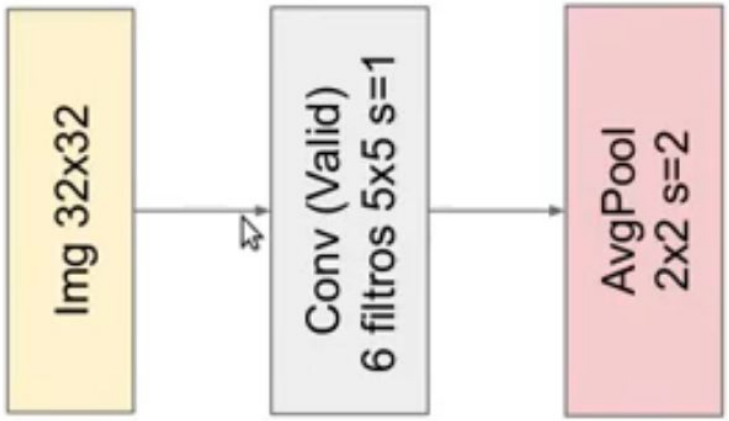
Recapitulando: Redes de convolução - Pooling



O. Rippel. Spectral Representations for Convolutional Neural Networks

Redes de Convolução: Lenet-5

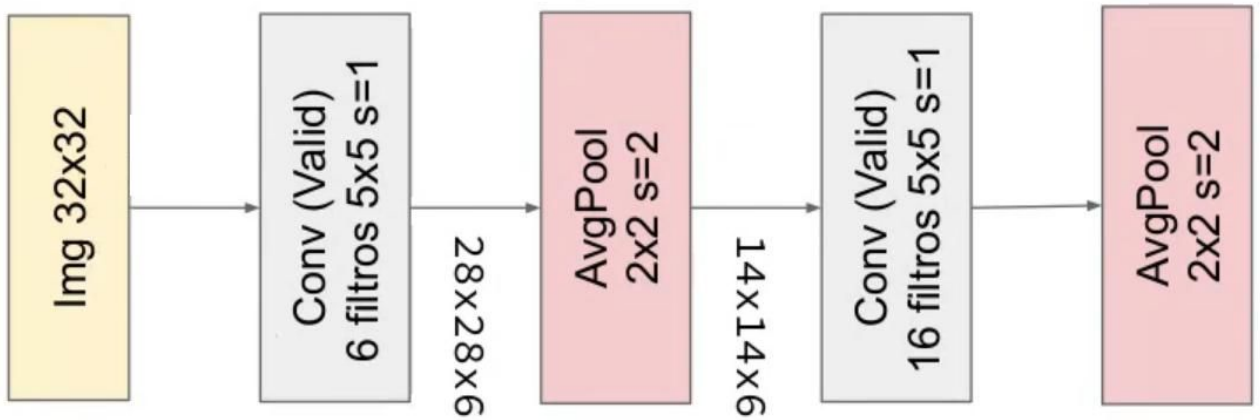
Rede aplicada a reconhecimento de dígitos escritos à mão.



LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). **Gradient-based learning applied to document recognition**. *Proceedings of the IEEE*, *86*(11).

Redes de Convolução: Lenet-5

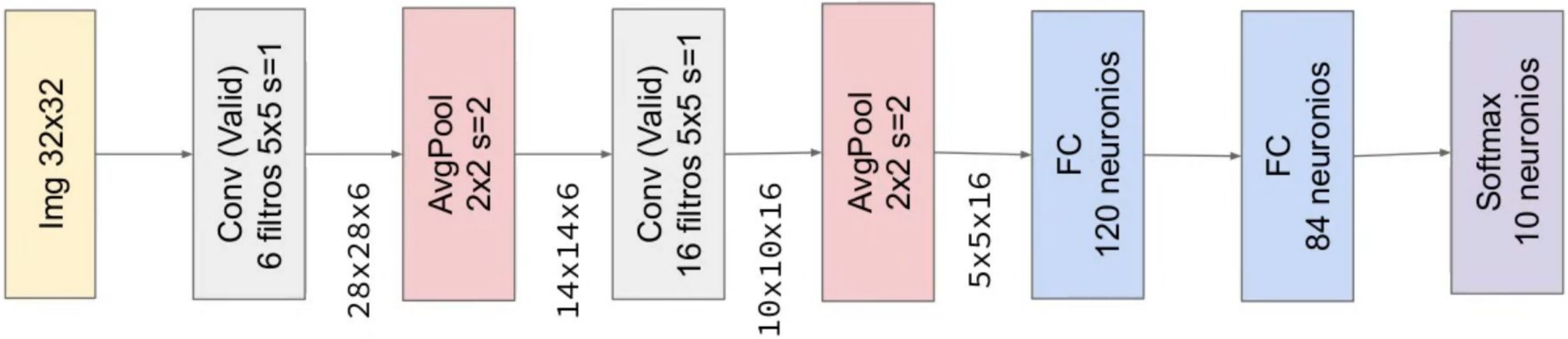
Rede aplicada a reconhecimento de dígitos escritos à mão.



LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). **Gradient-based learning applied to document recognition**. *Proceedings of the IEEE*, *86*(11).

Redes de Convolução: Lenet-5

Rede aplicada a reconhecimento de dígitos escritos à mão.



Aprox. 60 mil parametros

No artigo original, aplicou-se tangh ou sigmoid nas convoluções.
Atualmente, pode-se usar Relu com um bom resultado.
Da mesma forma, atualmente usa-se mais maxPooling ao invés de avgPooling

Redes de Convolução: Lenet-5

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 28, 28, 6)         156

 max_pooling2d (MaxPooling2  (None, 14, 14, 6)         0
 D)

 conv2d_1 (Conv2D)           (None, 10, 10, 16)        2416

 max_pooling2d_1 (MaxPoolin  (None, 5, 5, 16)          0
 g2D)

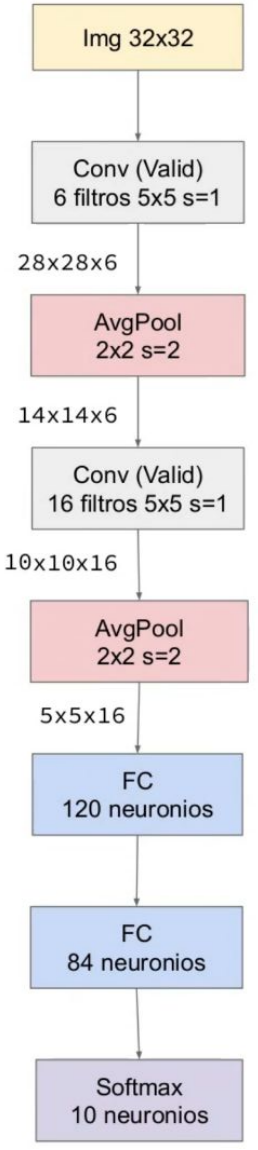
 flatten (Flatten)           (None, 400)               0

 dense (Dense)               (None, 120)               48120

 dense_1 (Dense)             (None, 84)                10164

 dense_2 (Dense)             (None, 10)                850

=================================================================
Total params: 61706 (241.04 KB)
Trainable params: 61706 (241.04 KB)
Non-trainable params: 0 (0.00 Byte)
```



Arquitetura convolucional da AlexNet

- Na hora certa (2012)…
  - Antes do AlexNet, redes MLP e CNNs se saíam bem para coleções como MNIST, de classificação de caracteres escritos à mão.
  - Entretanto, para outros problemas reais, era necessário outro benchmark:
    - ImageNet (22k classes e 15 milhões de imagens) - 2010
  - Poder computacional:
    - Artigo usou a GPU GTX 580 (6 dias de treinamento)
  - Função de ativação:
    - Em 2010 Vinod Nair e Geoffrey E. Hinton introduziram as ReLU
      - Não linear eficaz e barata de computar em relação à sigmoid e tanh

Arquitetura convolucional da AlexNet

- As camadas…
  - Entrada 224x224x3
  - Primeiro filtro 11x11
  - Filtros convolucionais e pooling menores em camadas mais profundas
  - Ex.: 5x5, 3x3

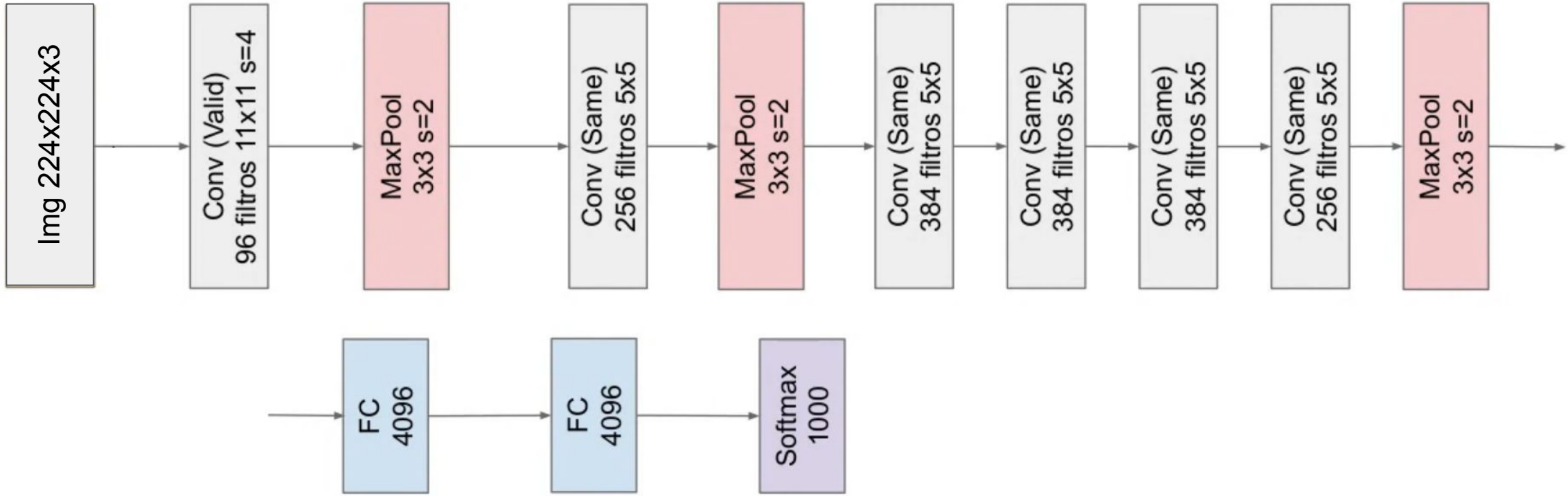
  - O que acontece se usarmos apenas filtros pequenos? (3x3, por exemplo?)

Arquitetura convolucional da AlexNet

- As camadas…
  - Entrada 224x224x3
  - Primeiro filtro 11x11
  - Filtros convolucionais e pooling menores em camadas mais profundas
  - Ex.: 5x5, 3x3

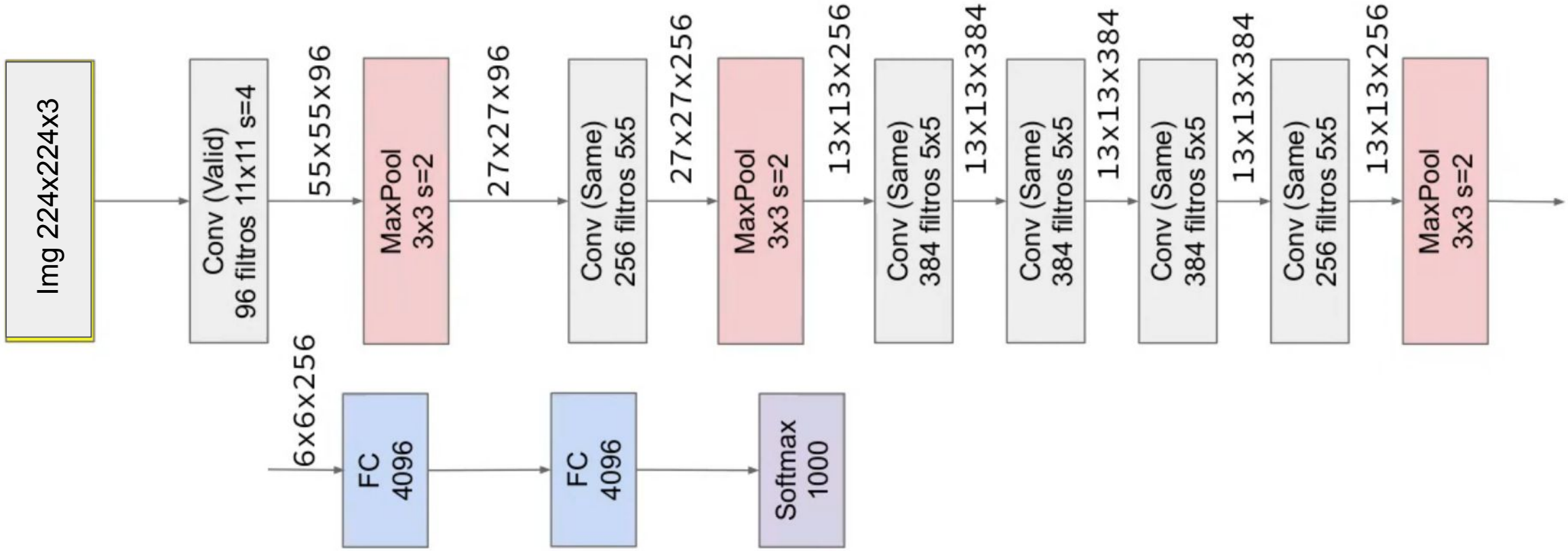
  - O que acontece se usarmos apenas filtros pequenos? (3x3, por exemplo?)
    - perde contexto espacial, detalhes
    - filtros maiores pegam “pedaços maiores” da matriz
      - mais pesos -> mais lento para treinar

Redes de Convolução: AlexNet



Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). **Imagenet classification with deep convolutional neural networks**. In *Advances in neural information processing systems*.

Redes de Convolução: AlexNet



Ativação das camadas Conv: Relu
Aprox. 60 milhões de parâmetros

Arquitetura convolucional da AlexNet

- AlexNet combina conceitos como:
  - Data Augmentation
  - Normalization (Local Response Normalization)
  - Dropout
  - “Pre-training” e “Fine-tunning”

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

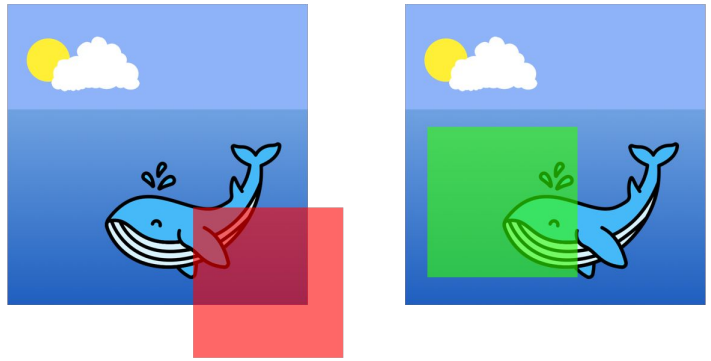
Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

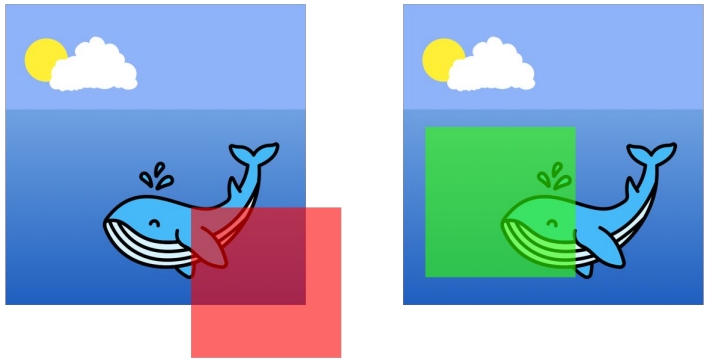
Arquitetura convolucional da AlexNet

- Além da arquitetura, o ponto do Data Augmentation é uma das contribuições mais importantes:
  - Data augmentation consiste em construir/distorcer artificial e aleatoriamente as imagens de treino, obtendo uma quantidade maior de imagens de treino.
    - Com uma imagem original (256x256), gera imagens reduzidas:
      - Todas de 224x224 (cropping)
      - Reflexões horizontais
      - Aumentou o treinamento em milhares de vezes



Arquitetura convolucional da AlexNet

- Além da arquitetura, o ponto do Data Augmentation é uma das contribuições mais importantes:
  - "The first form of data augmentation consists of generating image translations and horizontal reflections. We do this by extracting random 224 × 224 patches (and their horizontal reflections) from the 256×256 images and training our network on these extracted patches . **This increases the size of our training set by a factor of 2048**, though the resulting training examples are, of course, highly interdependent. **Without this scheme, our network suffers from substantial overfitting, which would have forced us to use much smaller networks. "**



Arquitetura convolucional da AlexNet

- Keras possui funções que distorcem as imagens de forma mais sofisticada. As funções de data augmentation de Keras permitem, por exemplo:
  - Deslocar a imagem horizontal ou verticalmente;
  - Espelhar a imagem horizontal ou verticalmente;
  - Rotacionar a imagem;
  - Mudar contraste da imagem (Keras chama de mudança de brilho o que na verdade seria contraste);
  - Redimensionar a imagem.

- Recursos interessantes:
  - https://keras.io/api/preprocessing/image/
  - https://machinelearningmastery.com/how-to-configure-image-data-augmentation-when-training-deep-learning-neural-networks/

Arquitetura convolucional da AlexNet

- Por exemplo especificando as distorções que queremos gerar (classe tensorflow.keras.preprocessing.image.ImageDataGenerator)

  - **datagen = ImageDataGenerator(width_shift_range=0.2, height_shift_range=0.2)**
    - indicam para deslocar aleatoriamente até 20% da largura e 20% da altura da imagem.
    - Em seguida, o método flow fornecerá um iterator que em cada chamada escolhe aleatoriamente batch_size imagens entre as imagens X:
  - **it = datagen.flow(X, batch_size=3, seed=None)**
    - Toda vez que o método it.next() é chamado, fornece batch de 3 imagens distorcidas:
  - **batch = it.next()**

Redes de Convolução: VGGNet

- *Main contributions:*
  - *“Thorough evaluation of networks of increasing depth using an architecture with very small (3 × 3) convolution filters”*
  - *“our representations generalise well to other datasets”*
- Um dos vencedores da competição “ImageNet Large Scale Visual Recognition Challenge” (2014)
- Também usa: normalização, dropout, training augmentation
- Diferentes versões (VGG-19, VGG-16…, VGG-X) onde X é a quantidade de camadas
- 4GPUS (Titan Black) - 3 semanas

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* **& Andrew Zisserman+**
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen,az}@robots.ox.ac.uk

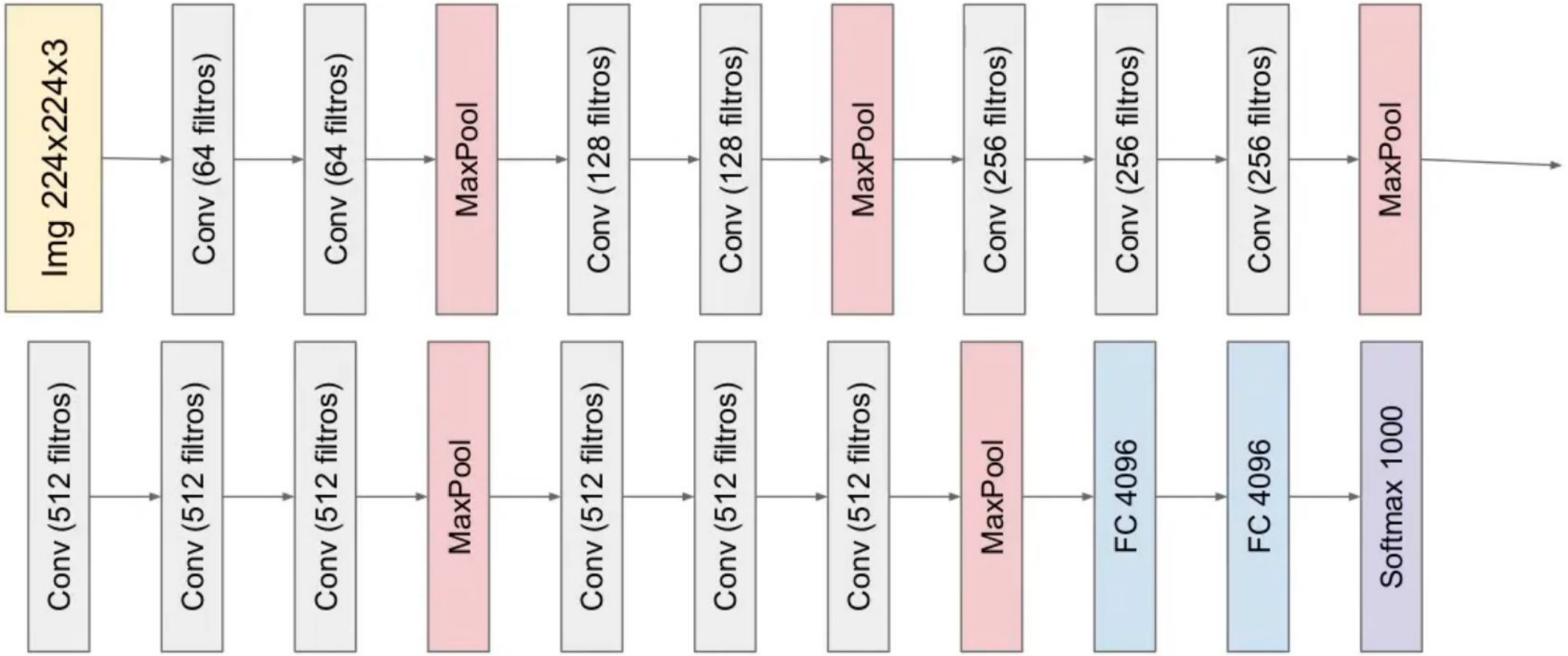
ABSTRACT

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16–19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

Redes de Convolução: VGGNet

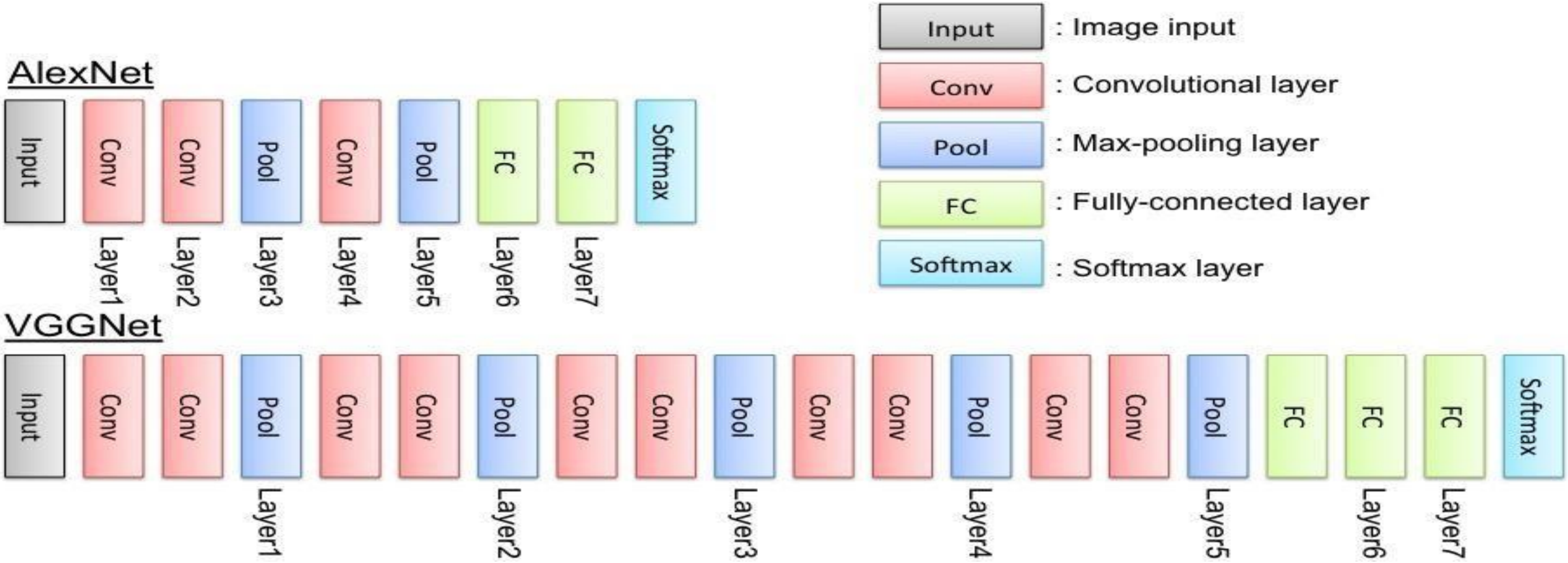
Conv=Camada Convolucional “same” com filtro 3x3 e s=1

MaxPool=MaxPool 2x2 s=2

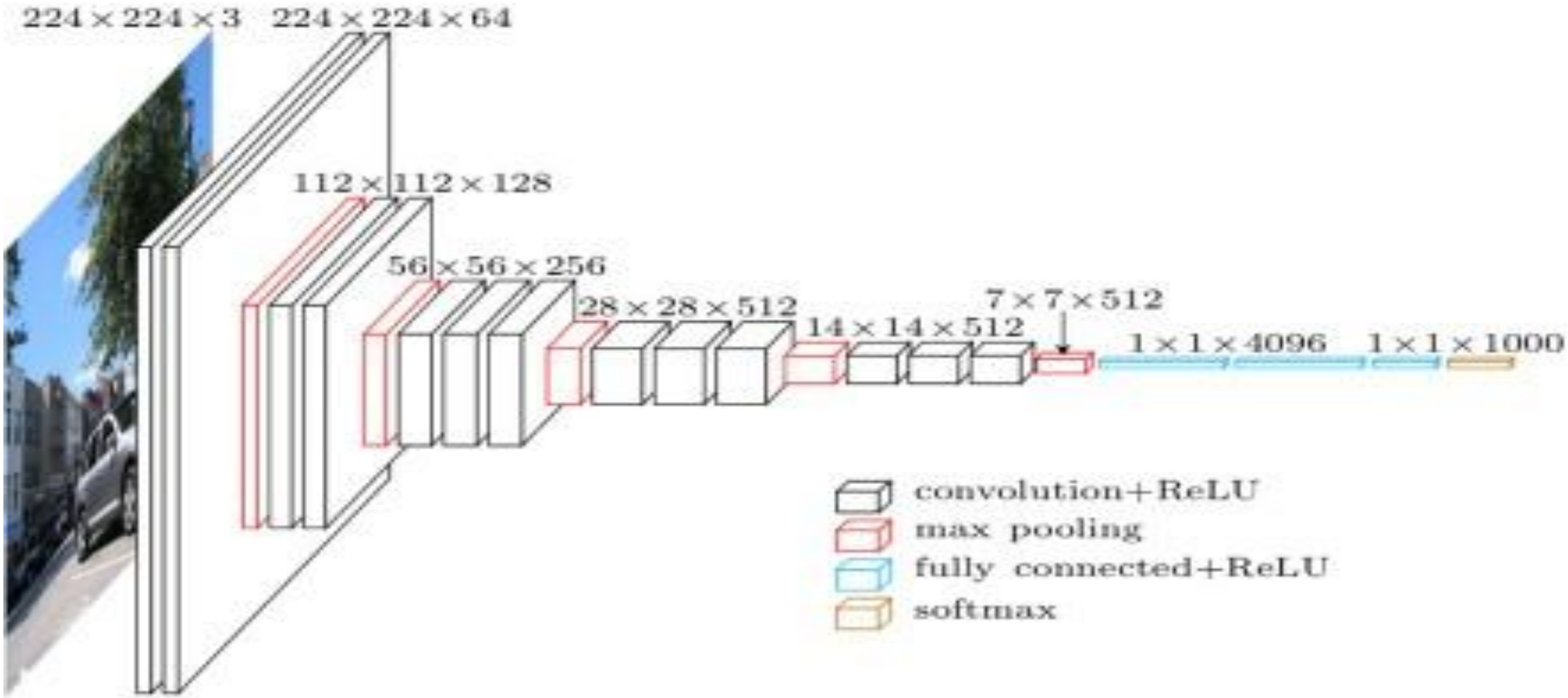


Simonyan, K., & Zisserman, A. (2014). **Very deep convolutional networks for large-scale image recognition**.

Características da VGGNet



VGGNet


224 × 224 × 3
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU
softmax

VGGNet

Porque o VGGNet virou um padrão de fato?

VGGNet

Porque o VGGNet virou um padrão de fato?

- Intuição: Os criadores pensaram em treinar uma rede de **propósito geral**!
  - Com filtros pequenos…
    - temos padrões menores, que podem aparecer em diversos contextos.
    - a rede monta essas características menores para padrões de alto nível.
      - Ex.: Para que um filtro que identifica uma orelha do gato (específico para o problema de classificação de animais) se posso montar essa orelha a partir de padrões menores?

VGGNet

Porque o VGGNet virou um padrão de fato?

- Intuição: Os criadores pensaram em treinar uma rede de **propósito geral**!
  - Com filtros pequenos…
    - temos padrões menores, que podem aparecer em diversos contextos.
    - a rede monta essas características menores para padrões de alto nível.
      - Ex.: Para que um filtro que identifica uma orelha do gato (específico para o problema de classificação de animais) se posso montar essa orelha a partir de padrões menores?
- Logo, utilizar uma VGGNet pré-treinada no Imagenet para um outro problema é justificável.
  - Hoje, uma das primeiras opções testadas em aplicações práticas de classificação de imagens.
  - Hoje, útil em situações onde há poucos recursos computacionais.
  - Na seara de aplicação: do VGGNet em diante, sempre usar os modelos pré-treinados (a não ser que você seja dono de um bom cluster).

GoogLeNet

- Criada por pesquisadores da google em 2014.
- Arquitetura Inception "Going Deeper with Convolutions"
  - "Uma rede neural dentro de uma camada"
- Superou o VGG em efetividade (ILSVRC14).
- Principais motivações:
  - Aproveitar a tendência das “very deep convolutional networks”
  - Cobrir diferentes níveis de informações espalhadas localmente.
  - Explorar complementaridade de informações em diferentes filtros de convolução.
  - Aumentar a eficiência da computação:
    - Redução da dimensionalidade

Rethinking the Inception Architecture for Computer Vision

Christian Szegedy
Google Inc.
szegedy@google.com

Vincent Vanhoucke
vanhoucke@google.com

Sergey Ioffe
sioffe@google.com

Jonathon Shlens
shlens@google.com

Zbigniew Wojna
University College London
zbigniewwojna@gmail.com

Abstract

Convolutional networks are at the core of most state-of-the-art computer vision solutions for a wide variety of tasks. Since 2014 very deep convolutional networks started to become mainstream, yielding substantial gains in various benchmarks. Although increased model size and computational cost tend to translate to immediate quality gains for most tasks (as long as enough labeled data is provided for training), computational efficiency and low parameter count are still enabling factors for various use cases such as mobile vision and big-data scenarios. Here we are exploring ways to scale up networks in ways that aim at utilizing the added computation as efficiently as possible by suitably factorized convolutions and aggressive regularization. We benchmark our methods on the ILSVRC 2012 classification challenge validation set demonstrate substantial gains over the state of the art: 21.2% top-1 and 5.6% top-5 error for single frame evaluation using a network with a computational cost of 5 billion multiply-adds per inference and with using less than 25 million parameters. With an ensemble of 4 models and multi-crop evaluation, we report 3.5% top-5 error and 17.3% top-1 error.

larly high performance in the 2014 ILSVRC [16] classification challenge. One interesting observation was that gains in the classification performance tend to transfer to significant quality gains in a wide variety of application domains. This means that architectural improvements in deep convolutional architecture can be utilized for improving performance for most other computer vision tasks that are increasingly reliant on high quality, learned visual features. Also, improvements in the network quality resulted in new application domains for convolutional networks in cases where AlexNet features could not compete with hand engineered, crafted solutions, e.g. proposal generation in detection[4].

Although VGGNet [18] has the compelling feature of architectural simplicity, this comes at a high cost: evaluating the network requires a lot of computation. On the other hand, the Inception architecture of GoogLeNet [20] was also designed to perform well even under strict constraints on memory and computational budget. For example, GoogleNet employed only 5 million parameters, which represented a 12× reduction with respect to its predecessor AlexNet, which used 60 million parameters. Furthermore, VGGNet employed about 3x more parameters than AlexNet.

GoogLeNet: Princípios de design

1) *"Avoid representational bottlenecks, especially early in the network"*
   - *Camadas mais baixas devem evitar perda de informação nos mapas de ativação.*
2) *"Higher dimensional representations are easier to process locally within a network."*
   - Filtros de convolução menores para processar imagens grandes.
3) *"Spatial aggregation can be done over lower dimensional embeddings"*
   - *Antes de usar um filtro maior, "compactar" as informações de entrada dos múltiplos mapas de ativação de entrada para esse filtro.*
4) *"Balance the width and depth of the network."*

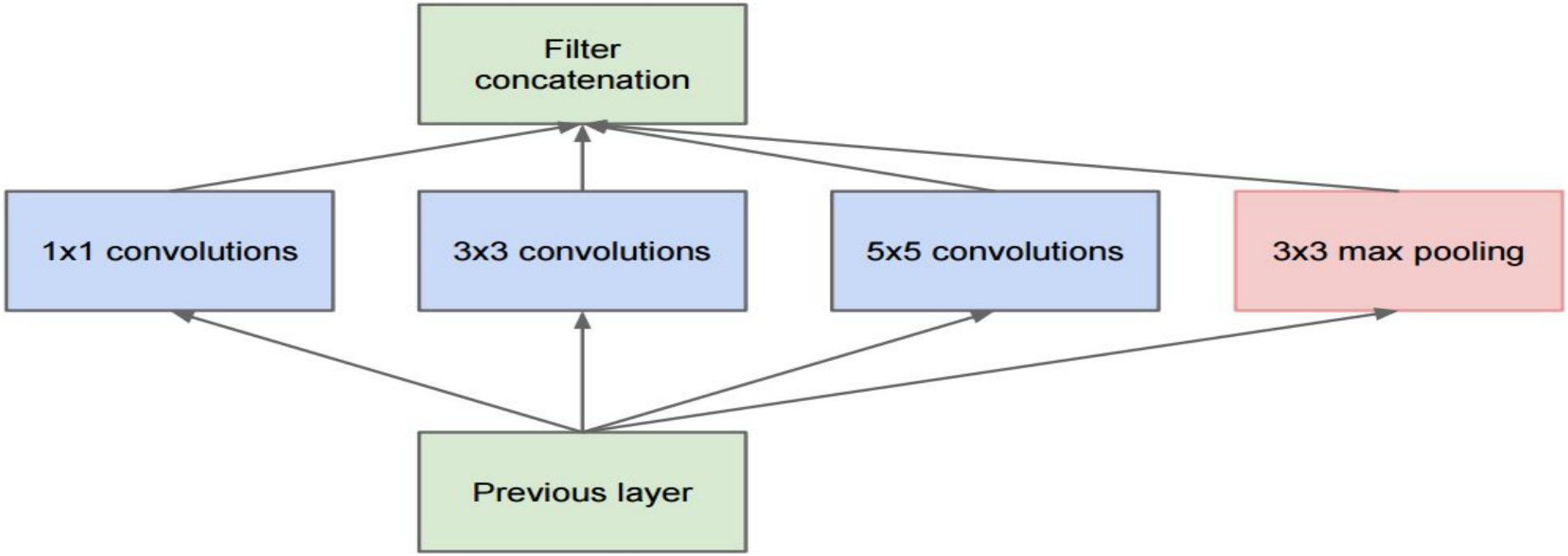
GoogLeNet

- Uso paralelo de filtros pequenos (1x1, 3x3, 5x5):
  - Adicionam informações complementares sobre localidade
  - Redução do custo computacional
- Inclui o uso de RELU
- 22 camadas (mais profunda e mais larga)

http://iamaaditya.github.io/2016/03/one-by-one-convolution/
https://www.coursera.org/lecture/convolutional-neural-networks/networks-in-networks-and-1x1-convolutions-ZTb8x

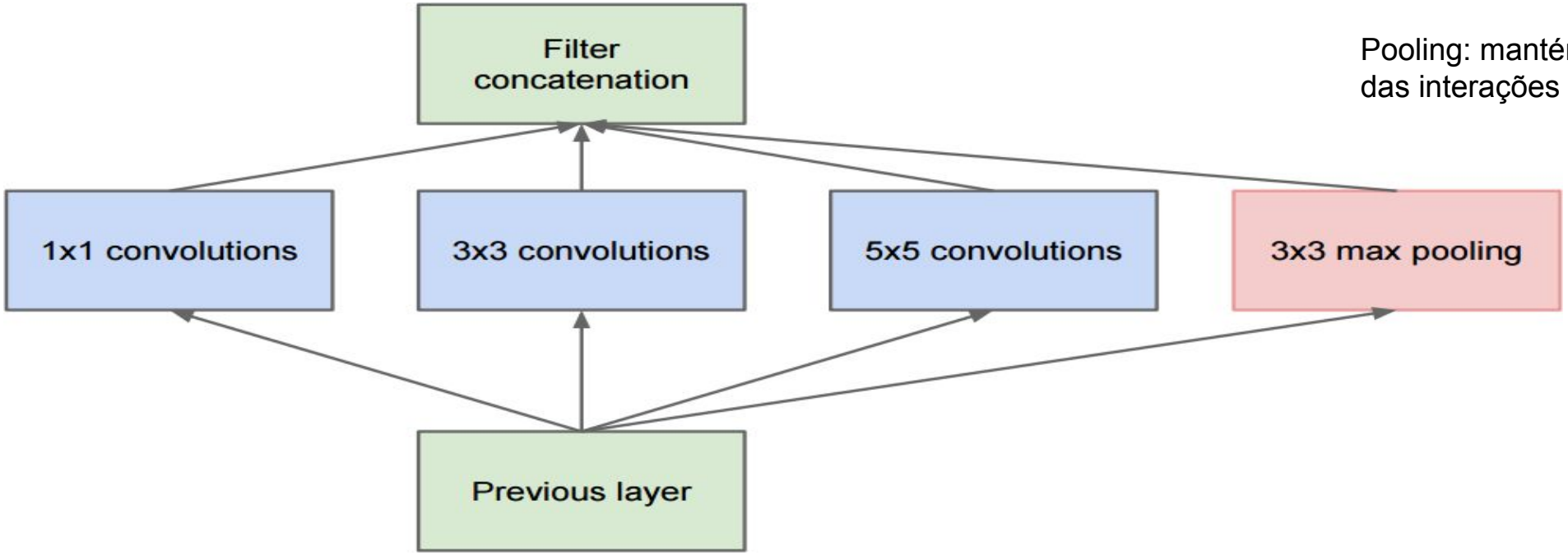
GoogLeNet

- Inception Module v1
  - Filtros convolucionais em paralelo com informações locais complementares



GoogLeNet

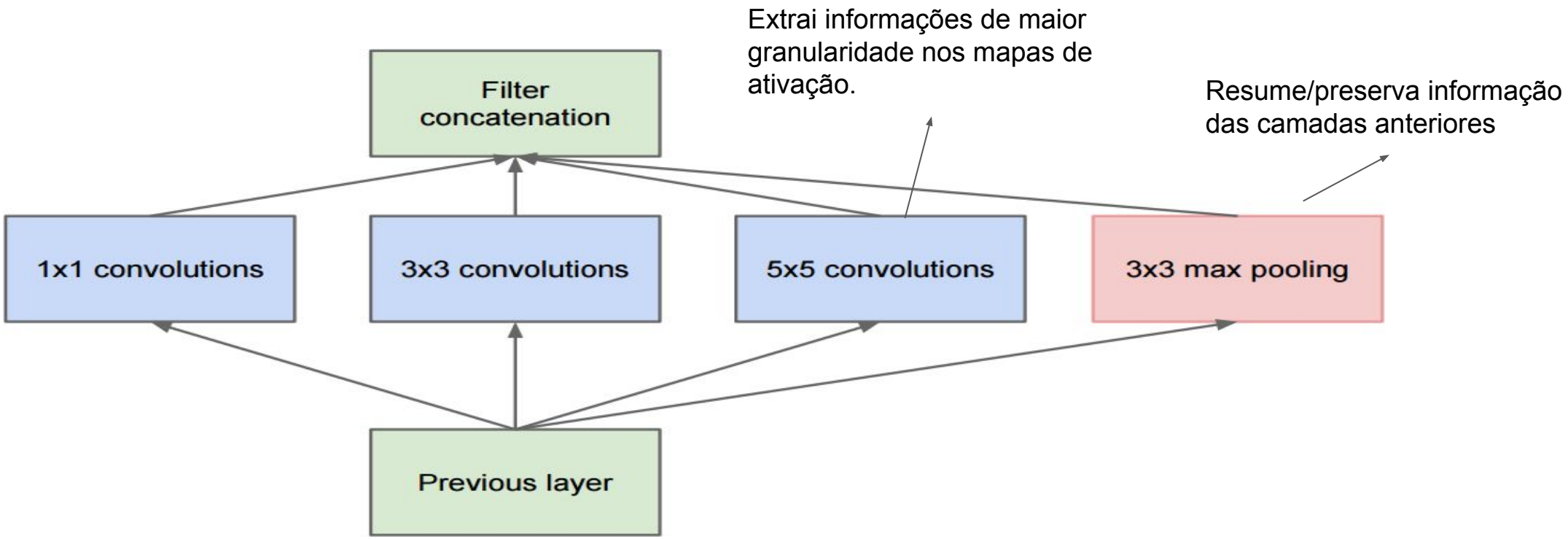
- Inception Module v1
  - Filtros convolucionais em paralelo com informações locais complementares



Pooling: mantém a informação das interações anteriores

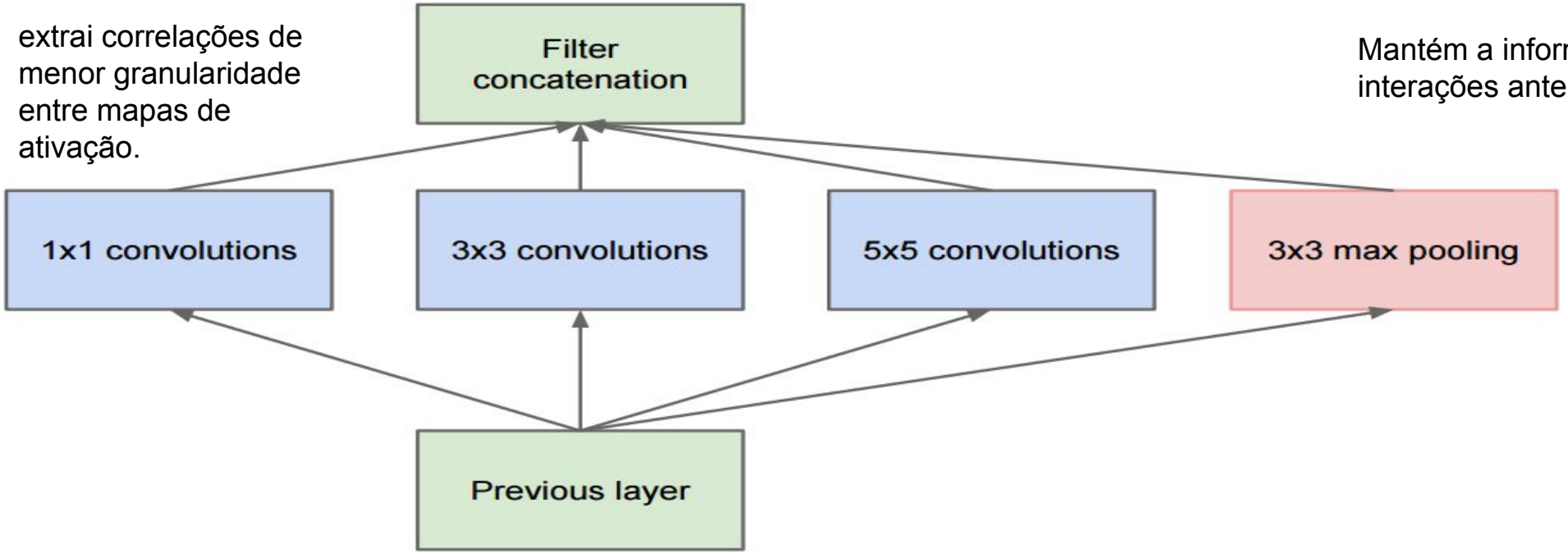
GoogLeNet

- Inception Module v1
  - Filtros convolucionais em paralelo com informações locais complementares



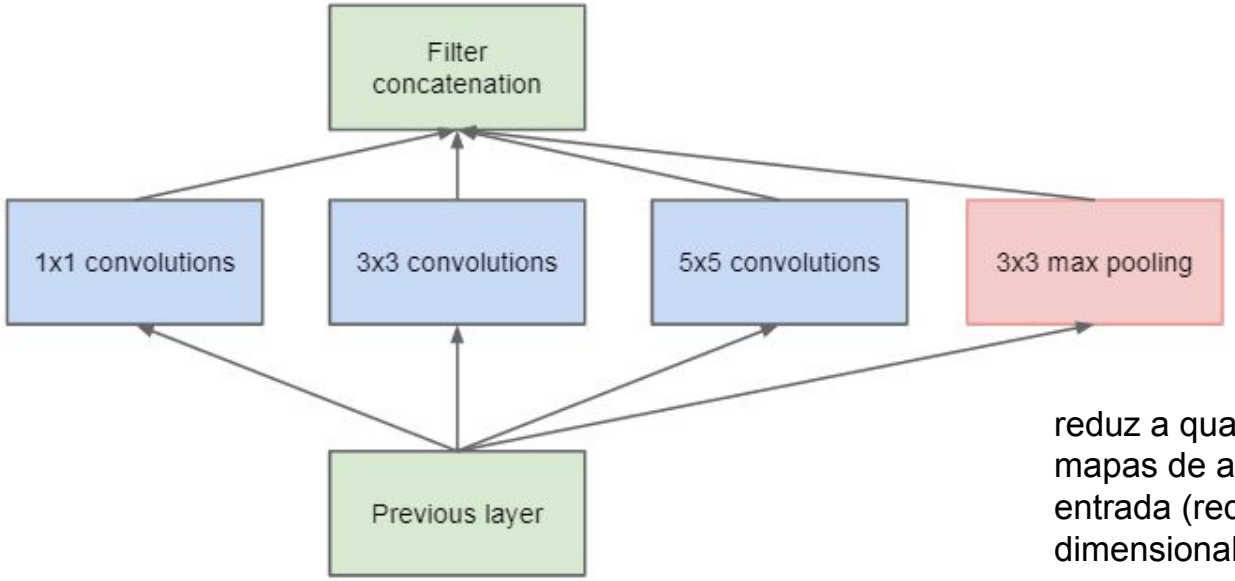
GoogLeNet

- Inception Module v1
  - Filtros convolucionais em paralelo com informações locais complementares

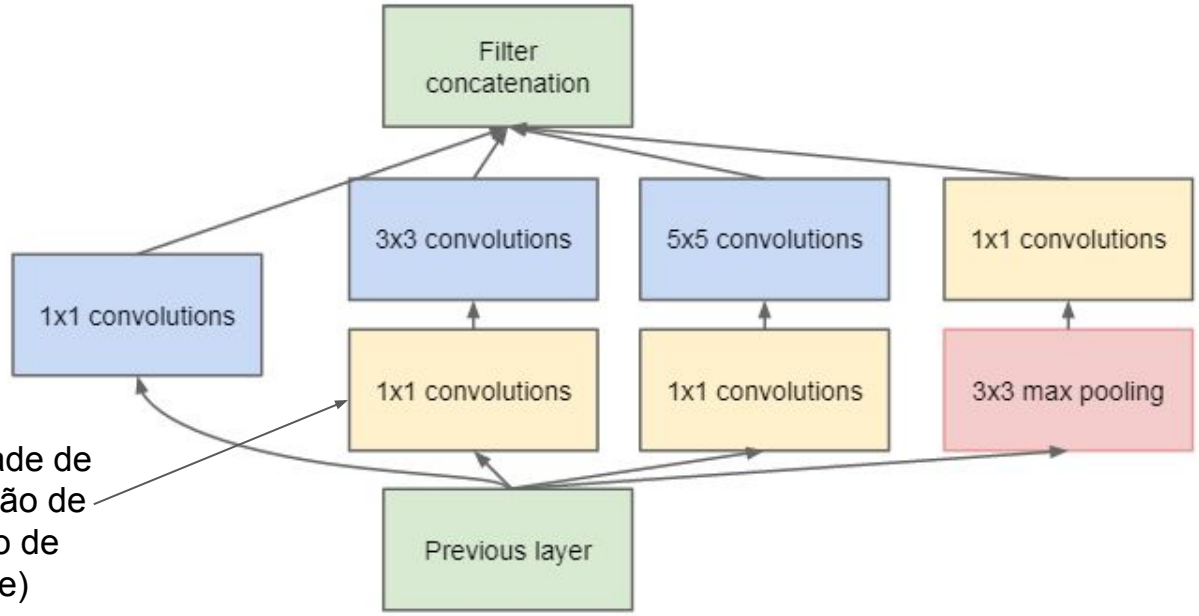


GoogLeNet

- Inception Module v1
  - Filtros convolucionais em paralelo com informações locais complementares
  - Usa filtros 1x1 para redução de dimensionalidade antes dos filtros 3x3 e 5x5
    - redução de custo com a redução do número de mapas de ativação.



(a) Inception module, naïve version



reduz a quantidade de mapas de ativação de entrada (redução de dimensionalidade)

(b) Inception module with dimension reductions

GoogLeNet

- Convolução 1x1:
  - Reduz o esforço computacional antes dos filtros 3x3 e 5x5
  - Inclui o uso de RELU

Exemplo:

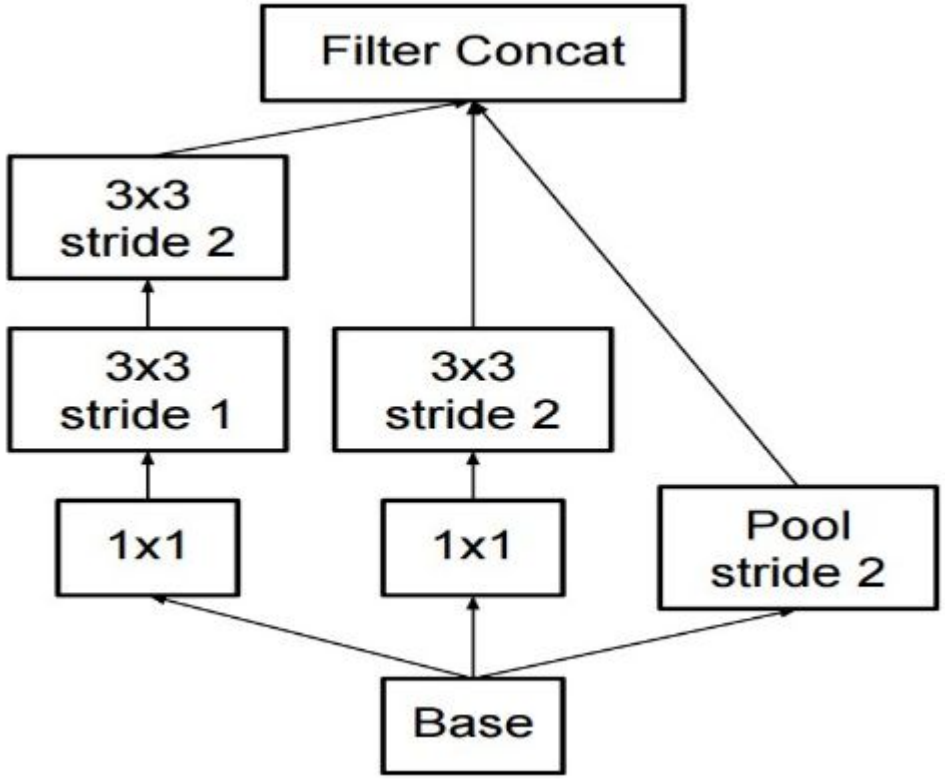
- Entrada 256 mapas de filtros 56x56
- Inserção de camada 1x1x64 (64 filtros 1x1)
- 56x56x256 -> 56x56x64 (redução de 4x)

https://www.coursera.org/lecture/convolutional-neural-networks/networks-in-networks-and-1x1-convolutions-ZTb8x

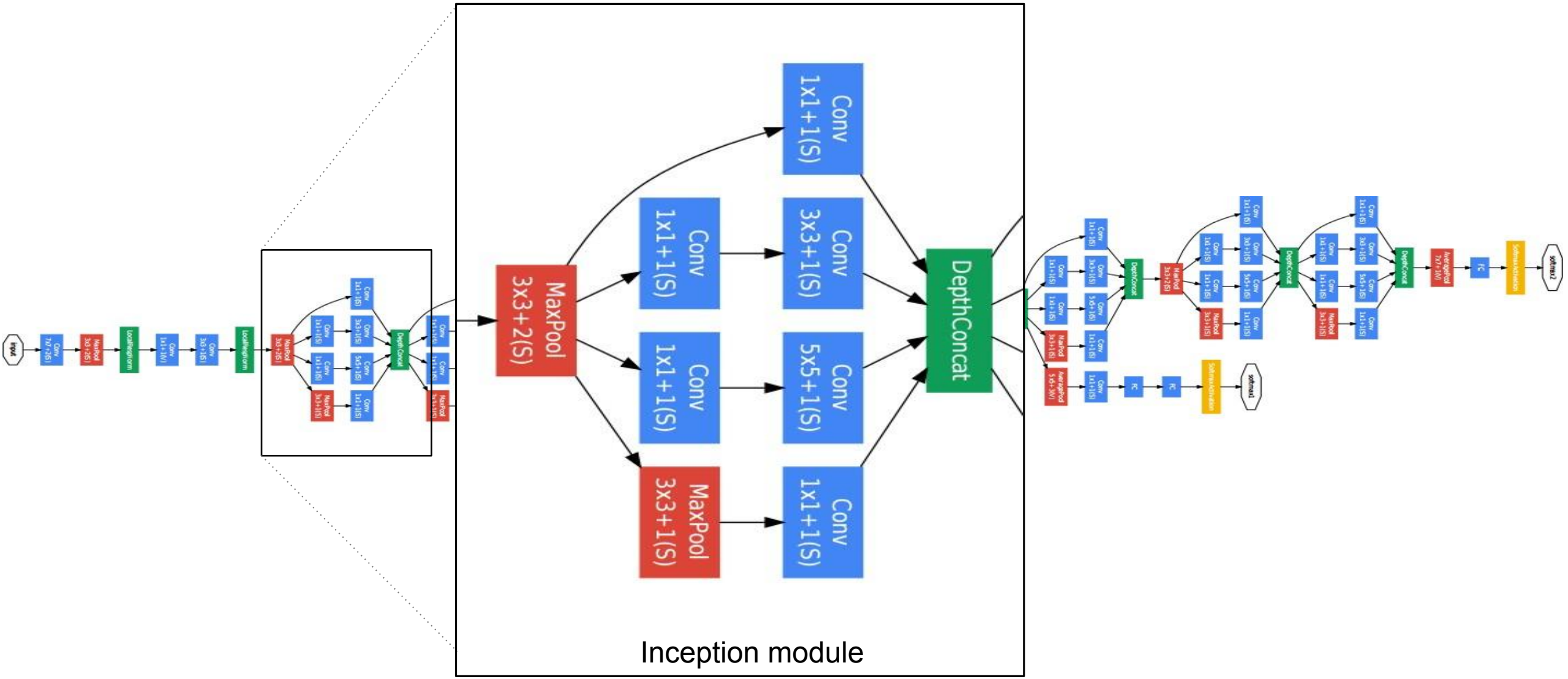
Inception (variações)

Inception v2 substitui as convoluções 5x5 por dois filtros 3x3 sucessivos:

- Avançando na exploração do princípio (3).

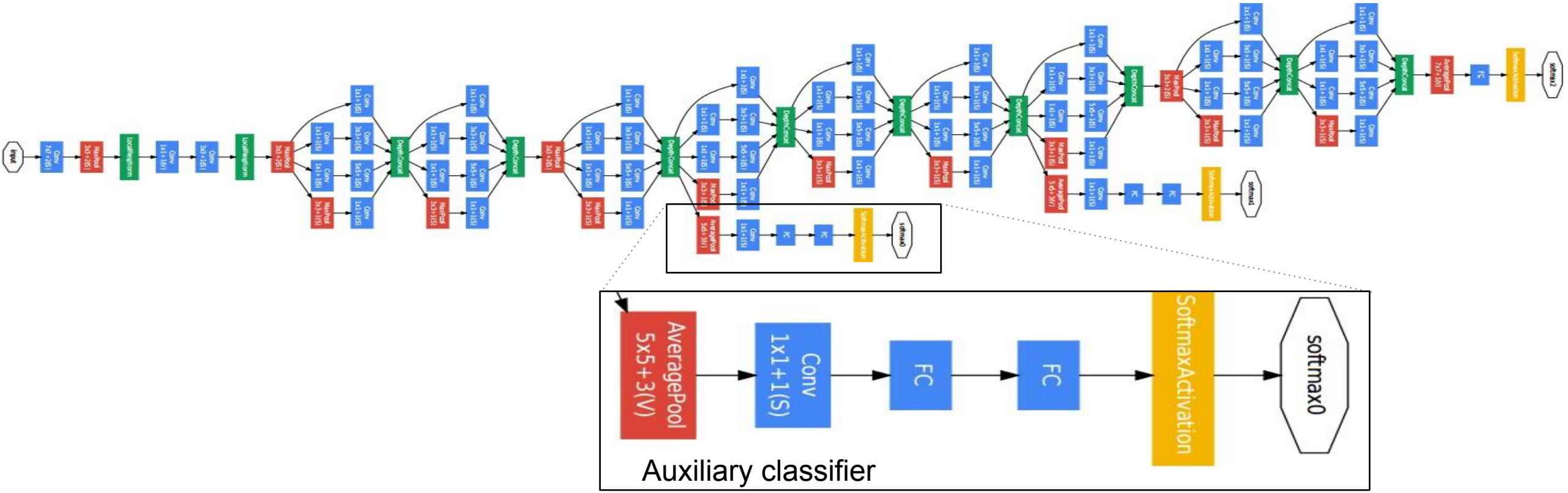


GoogLeNet



C. Szegedy et al., [Going deeper with convolutions](), CVPR 2015

GoogLeNet



C. Szegedy et al., Going deeper with convolutions, CVPR 2015

ResNet

Motivação:

- Aumentar a profundidade.
- Aprender dos “resíduos”:
  - “We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions”

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [41] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions[1], where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

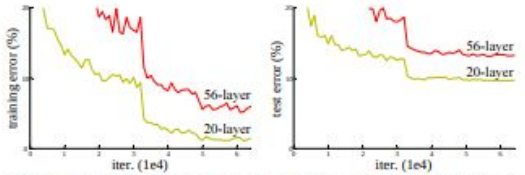


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

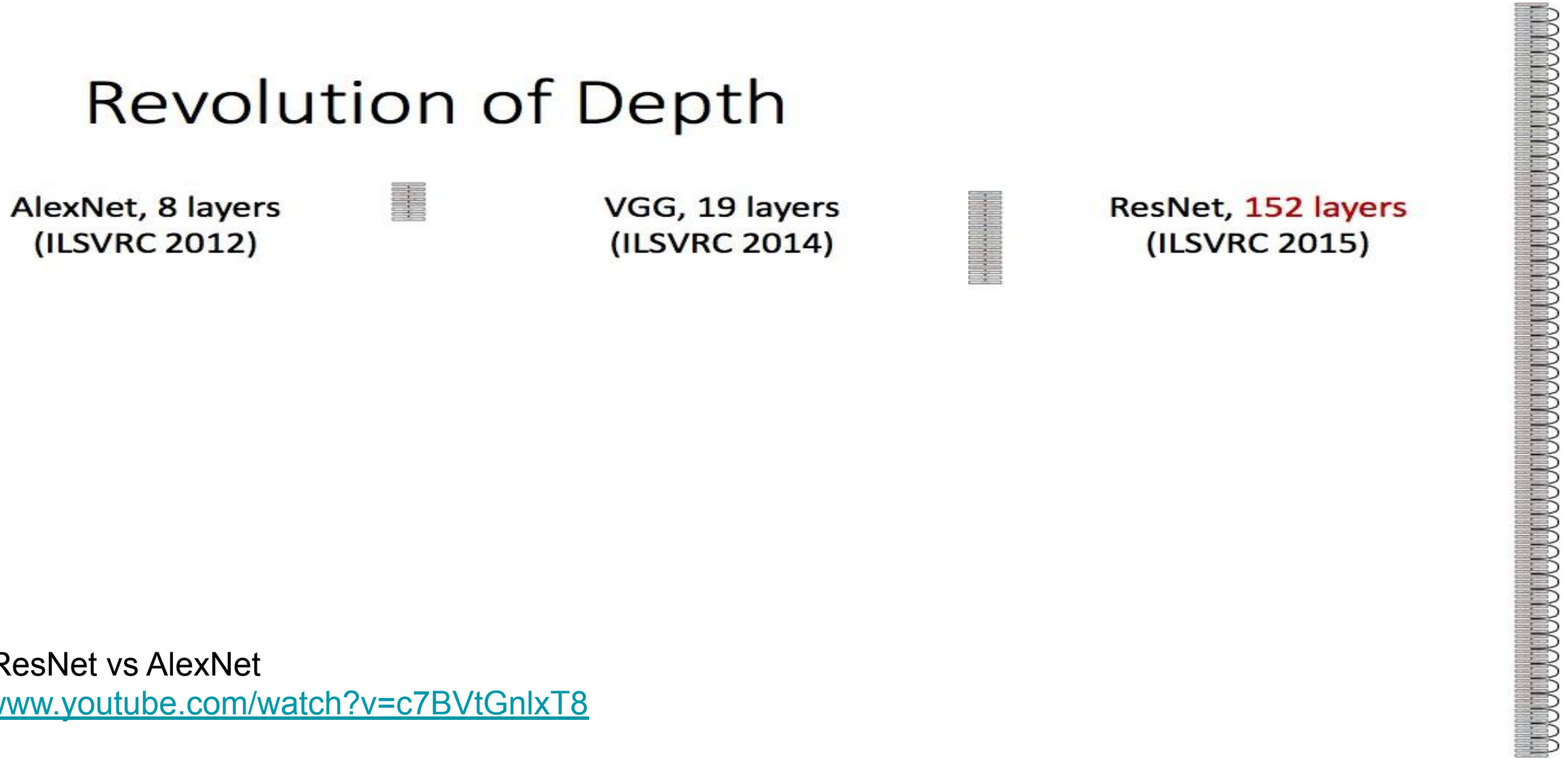
greatly benefited from very deep models.

Driven by the significance of depth, a question arises: *Is learning better networks as easy as stacking more layers?* An obstacle to answering this question was the notorious problem of vanishing/exploding gradients [1, 9], which hamper convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [23, 9, 37, 13] and intermediate normalization layers [16], which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with backpropagation [22].

When deeper networks are able to start converging, a *degradation* problem has been exposed: with the network

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, Deep Residual Learning for Image Recognition, CVPR 2016

ResNet

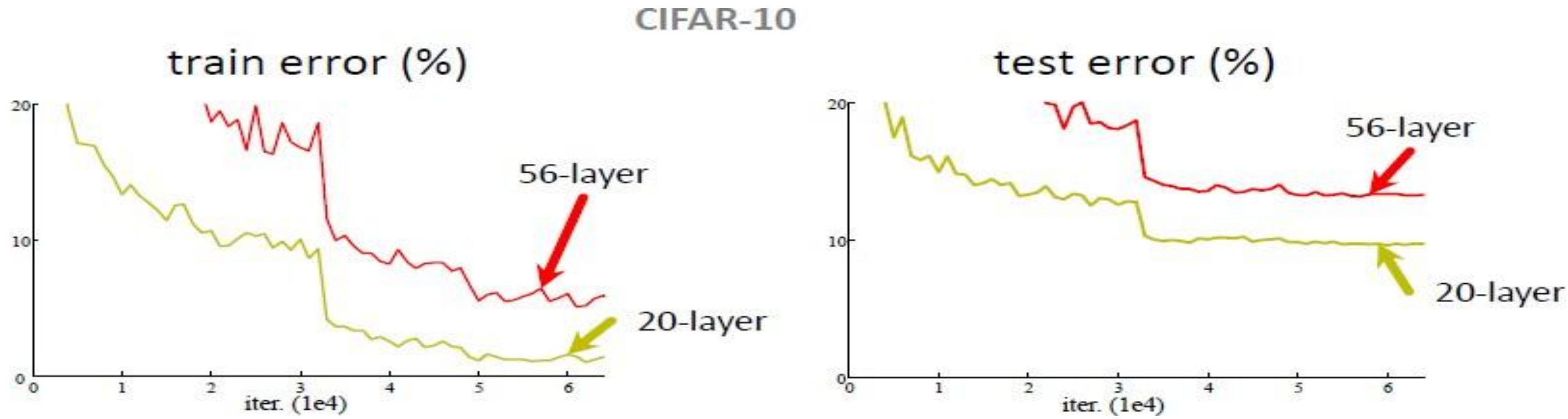


Vídeo: ResNet vs AlexNet
https://www.youtube.com/watch?v=c7BVtGnlxT8

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, Deep Residual Learning for Image Recognition, CVPR 2016

Problema:

- Até o momento: empilhar convoluções 3x3 funciona bem até 20~30 camadas
- Com mais camadas: propagação de ruído das camadas baixas, vanishing/exploding gradients.



Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, Deep Residual Learning for Image Recognition, CVPR 2016

Problema:

- Para problemas mais complexos, podemos pensar em
  - Mais camadas de convolução
  - Não lineariedade

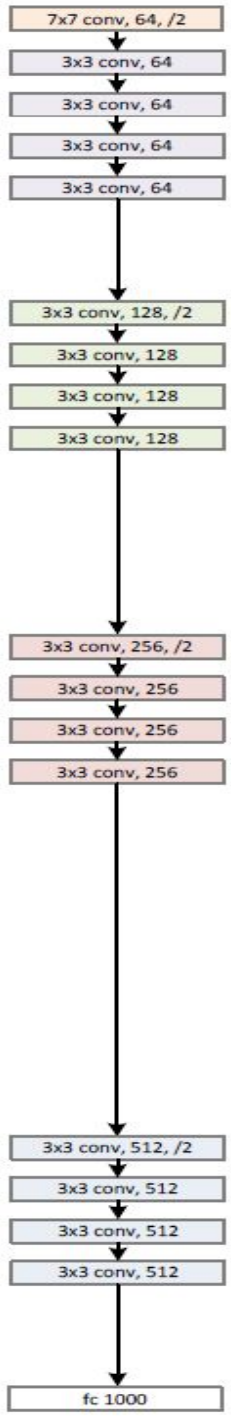
Entretanto, quanto maior a profundidade:

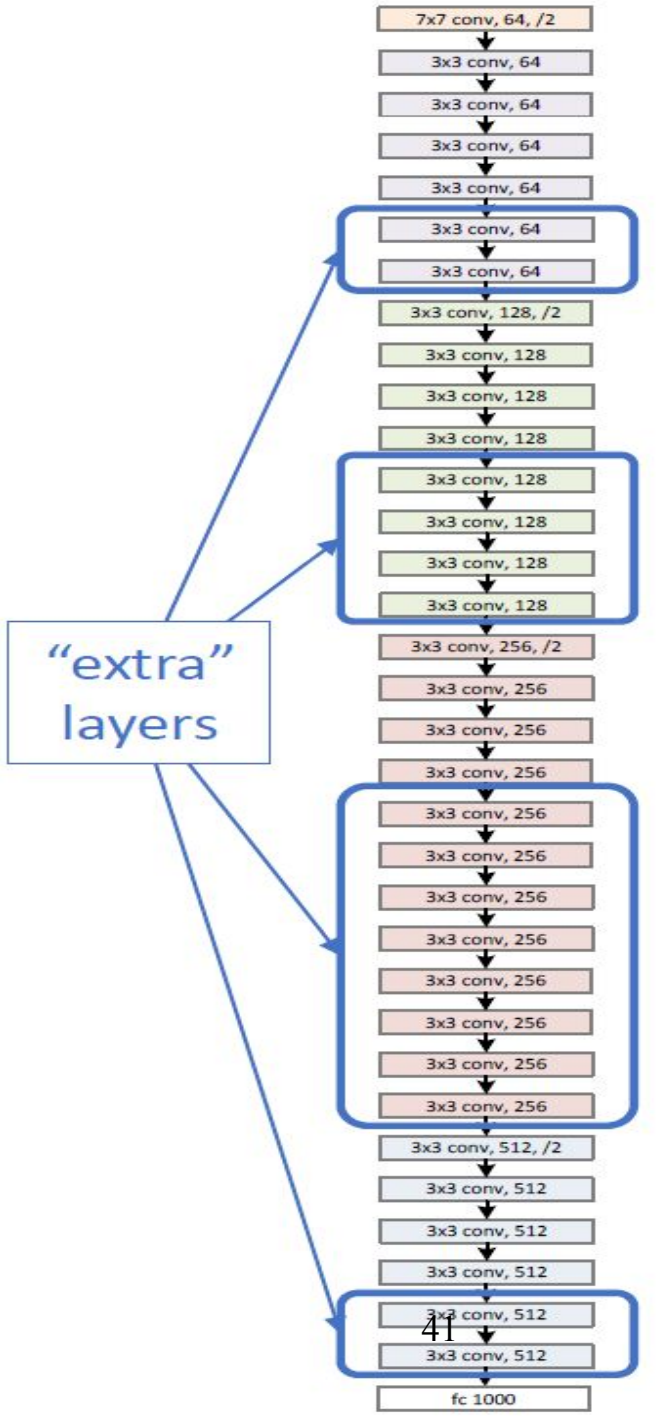
- O problema que tínhamos "resolvido" volta:
  - vanishing/exploding gradients problem
    - ocorre com sucessivas multiplicações de valores menores que 1 ou maiores que 1.

ResNet

Intiuição:

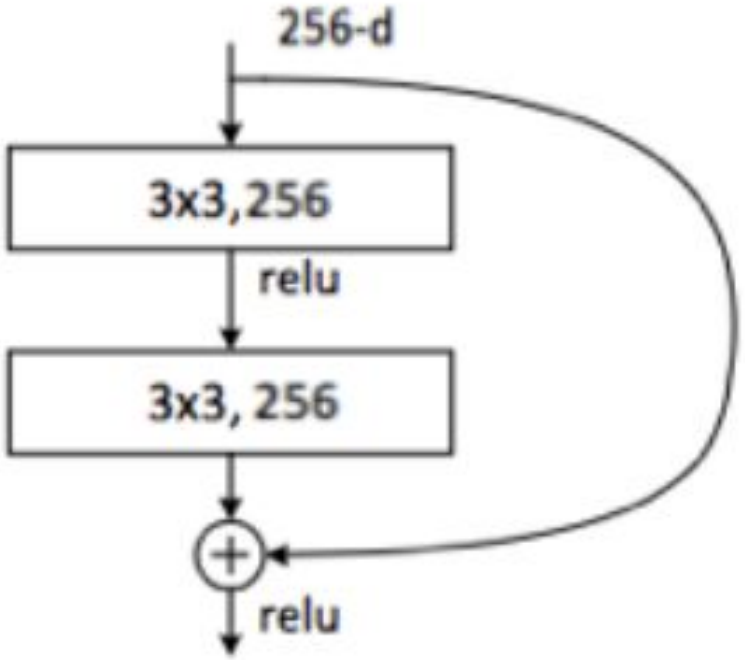
- Para uma nova camada, recebo o processamento (possivelmente ruidoso) da camada anterior.
- Hipótese:
  - Se as camadas extras forem de alguma forma um mapeamento de direto para próximas camadas, erros de treinamento não aumentarão, e a propagação do gradiente pode pular direto para camada seguinte.





ResNet

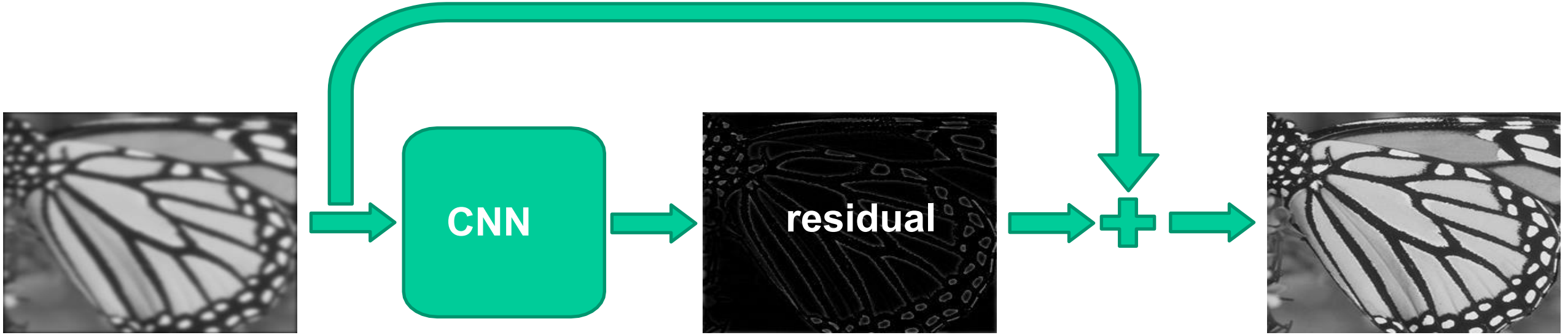
- Módulo residual
  - Introduz conexões de “atalho”



Exemplo de bloco residual

ResNet

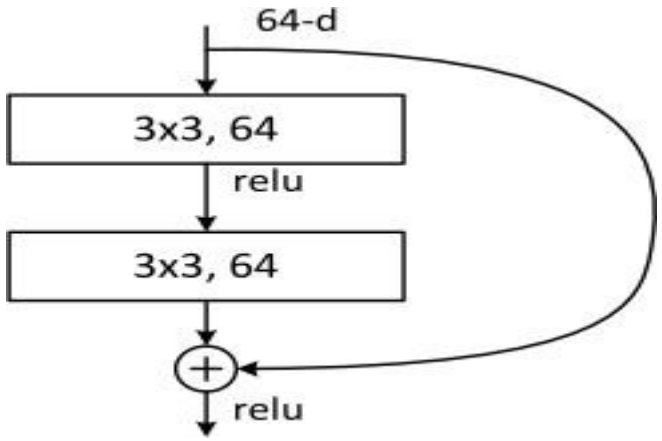
- Explicitamente permitimos que essas camadas se encaixem em um mapeamento residual.
- Hipótese de que é mais fácil otimizar a saída do bloco residual do que otimizar o mapeamento original.



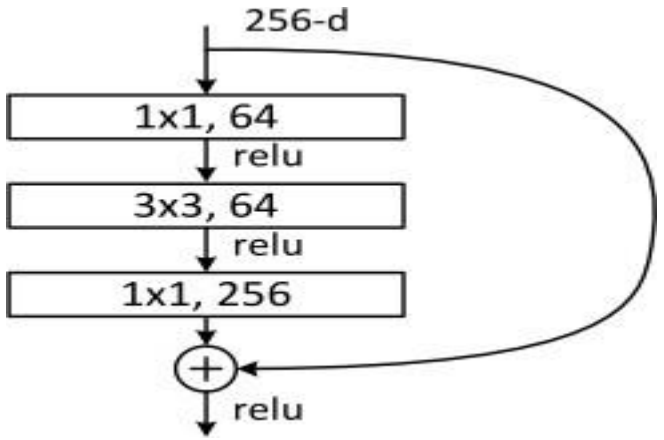
ResNet

Bloco Residual

- Simples
- Livre de parâmetro



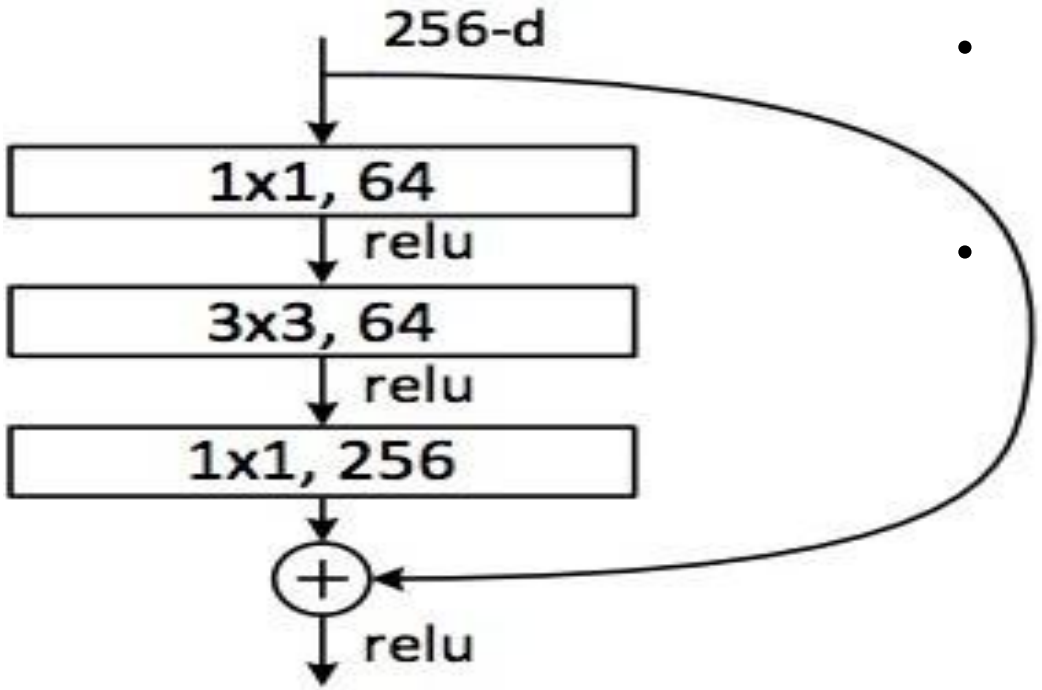
Bloco “naïve”



Bloco “bottleneck”
(ResNet-50/101/152)

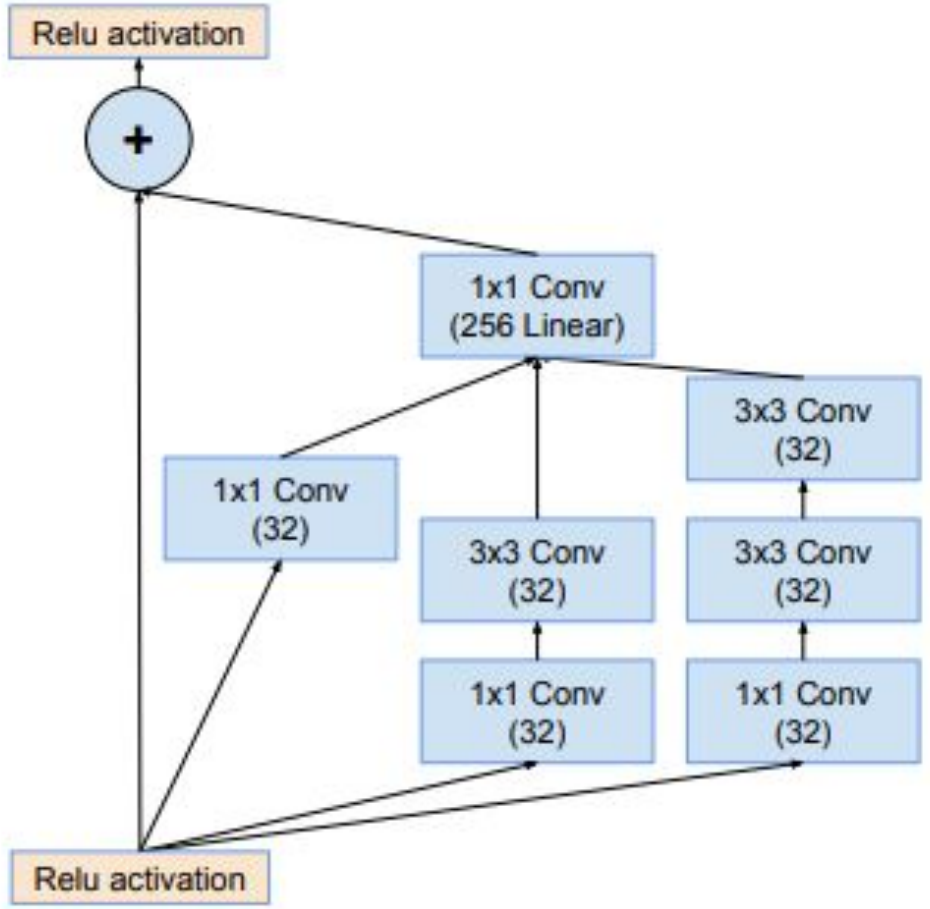
ResNet

Módulo “bottleneck”



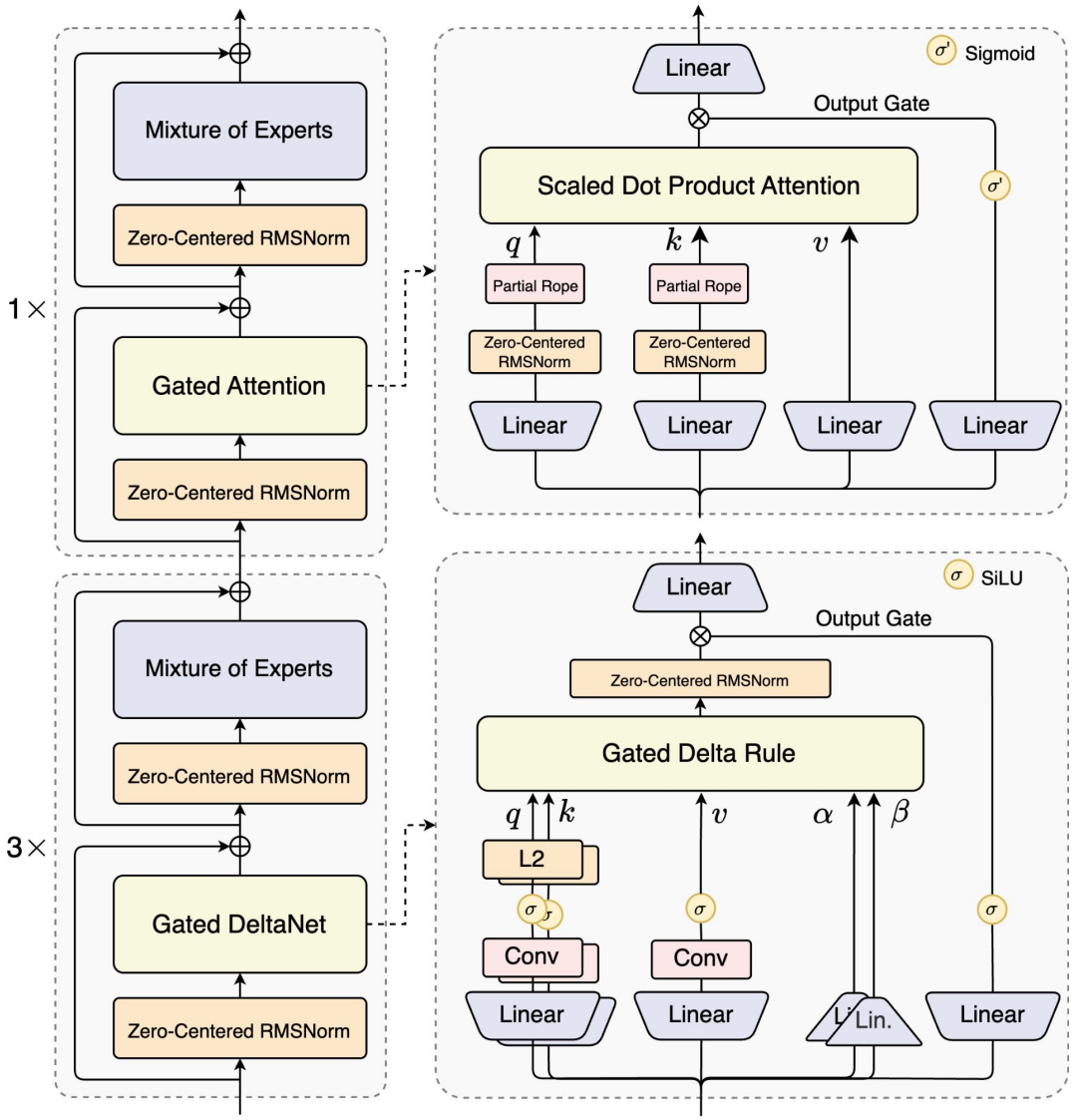
- Com convoluções 3x3, 256 mapas de entrada e 256 mapas de saída:
  256 x 256 x 3 x 3 ~ 600K
- Com convoluções 1x1 para reduzir de 256 para 64 mapas, seguidos por convoluções 3x3, seguidos por convoluções 1x1 para expandir para 256 mapas:
- 256 x 64 x 1 x 1 ~ 16K
  64 x 64 x 3 x 3 ~ 36K
  64 x 256 x 1 x 1 ~ 16K
  Total: ~70K

ResNet+GoogLeNet = InceptionV4



C. Szegedy, et. al. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. AAAI, 2017

Conexões residuais tiveram futuro?



Cao, R., et al. Qwen3-Coder-Next Technical Report., 2026. DOI: 10.48550/arXiv.2603.00729

Comparação

Desafio ImageNet: top-5 error

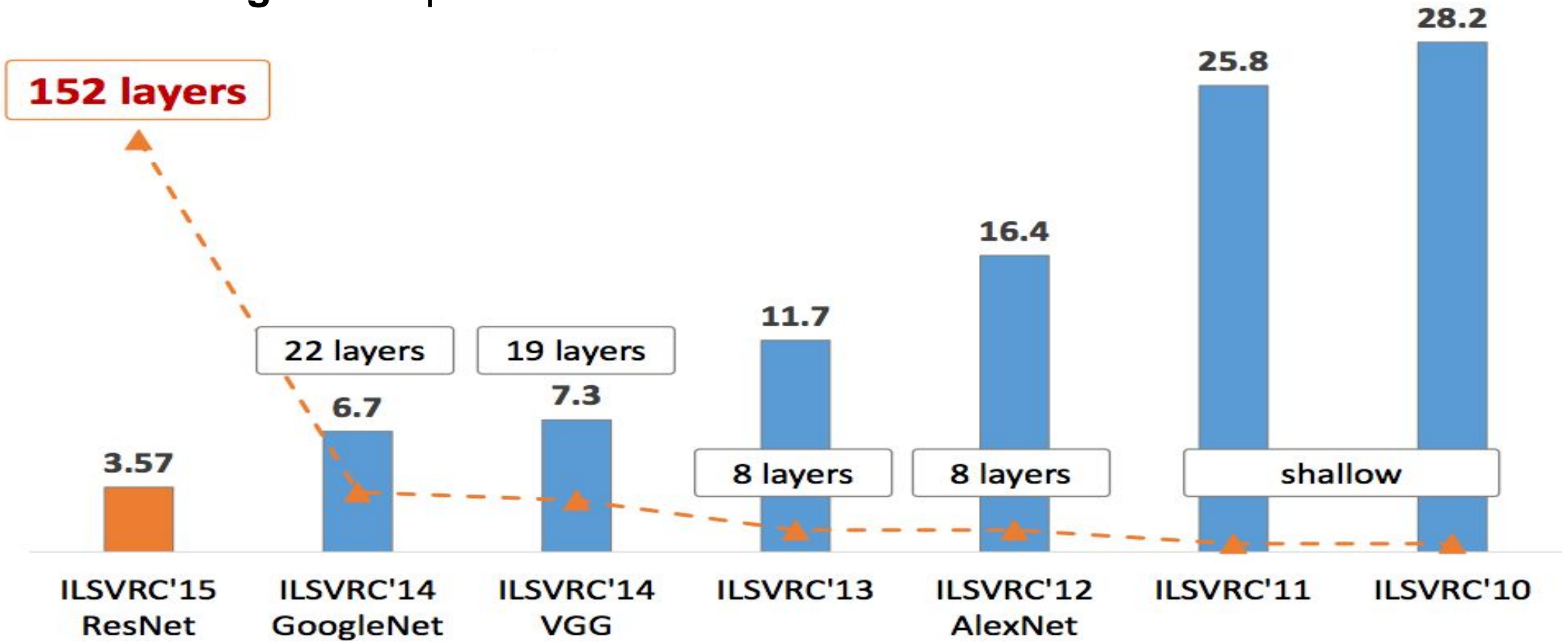


Figure source: Kaiming He

Redes de Convolução: DenseNet

Main contributions:

"Densely Connected Convolutional Networks": cada camada recebe como entrada os mapas de todas as camadas anteriores.

Estimula reuso de características e reduz parâmetros em relação a arquiteturas muito largas.

Ideia central: em vez de somar atalhos como na ResNet, concatenar características anteriores.

Densely Connected Convolutional Networks

Gao Huang*
Cornell University
gh349@cornell.edu

Zhuang Liu*
Tsinghua University
liuzhuang13@mails.tsinghua.edu.cn

Laurens van der Maaten
Facebook AI Research
lvdmaaten@fb.com

Kilian Q. Weinberger
Cornell University
kqw4@cornell.edu

Abstract

Recent work has shown that convolutional networks can be substantially deeper, more accurate, and efficient to train if they contain shorter connections between layers close to the input and those close to the output. In this paper, we embrace this observation and introduce the Dense Convolutional Network (DenseNet), which connects each layer to every other layer in a feed-forward fashion. Whereas traditional convolutional networks with L layers have L connections—one between each layer and its subsequent layer—our network has $\frac{L(L+1)}{2}$ direct connections. For each layer, the feature-maps of all preceding layers are used as inputs, and its own feature-maps are used as inputs into all subsequent layers. DenseNets have several compelling advantages: they alleviate the vanishing-gradient problem, strengthen feature propagation, encourage feature reuse, and substantially reduce the number of parameters. We evaluate our proposed architecture on four highly

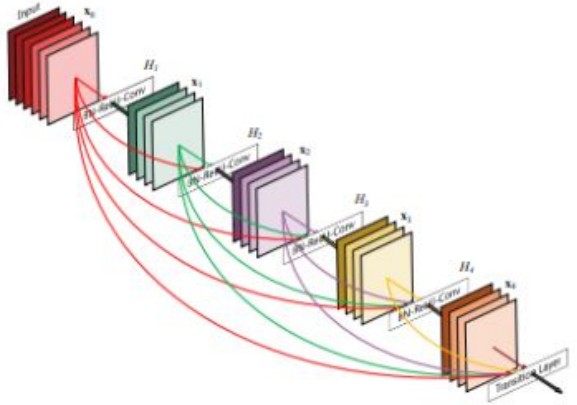


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

Huang, G.; Liu, Z.; van der Maaten, L.; Weinberger, K. Q. Densely Connected Convolutional Networks. CVPR 2017.

DenseNet: ideia central

Bloco denso:

Camada 1 produz novos mapas de ativação.

Camada 2 recebe entrada original + saída da camada 1.

Camada 3 recebe entrada original + saídas 1 e 2.

Consequência:

as camadas não precisam reaprender características já disponíveis.

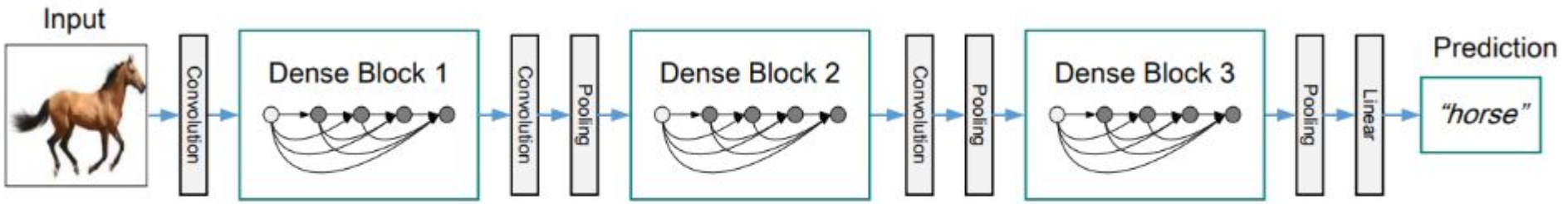
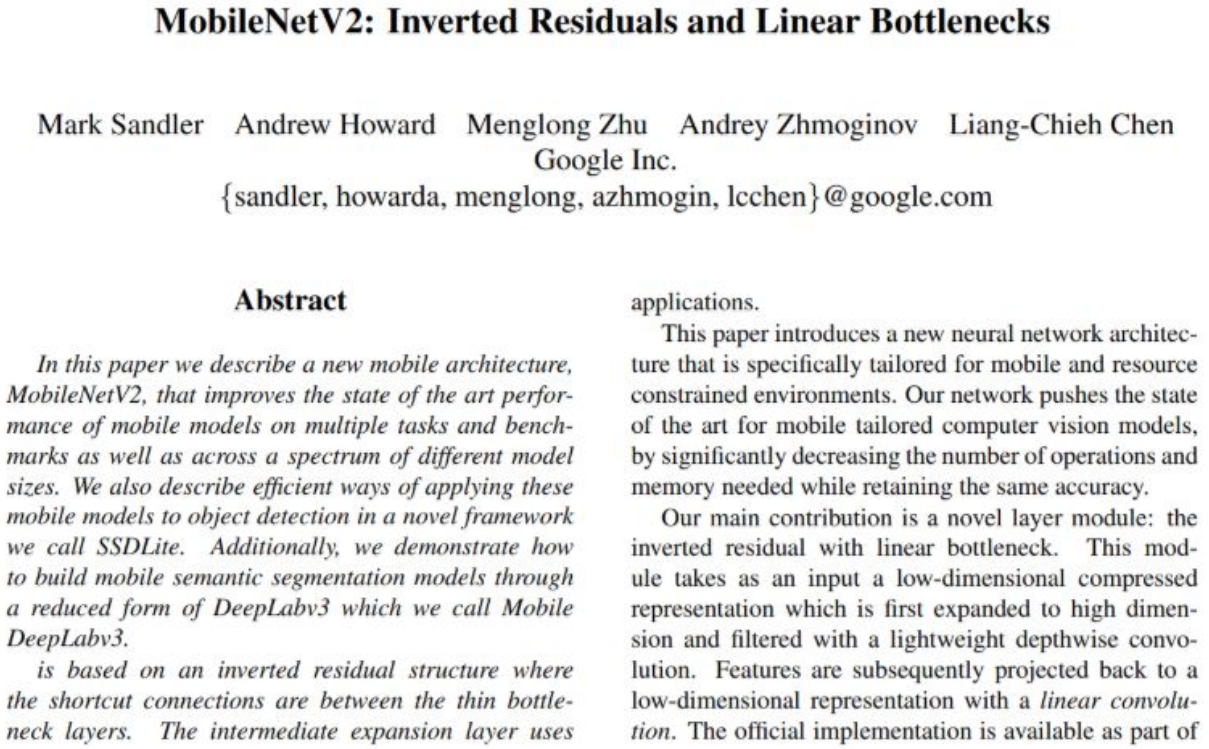


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

Na DenseNet, as conexões preservam características por concatenação, não por soma.

Redes de Convolução: MobileNetV2

- Propõe “inverted residuals” e “linear bottlenecks”.
- Expande temporariamente os canais, aplica convolução e projeta de volta para poucos canais.
- Usa conexão residual entre gargalos estreitos
- Ideia central: manter baixo custo, mas evitar perda de informação em representações de baixa dimensão.



MobileNetV2: Inverted Residuals and Linear Bottlenecks

Mark Sandler Andrew Howard Menglong Zhu Andrey Zhmoginov Liang-Chieh Chen
Google Inc.
{sandler, howarda, menglong, azhmogin, lcchen}@google.com

Abstract

In this paper we describe a new mobile architecture, MobileNetV2, that improves the state of the art performance of mobile models on multiple tasks and benchmarks as well as across a spectrum of different model sizes. We also describe efficient ways of applying these mobile models to object detection in a novel framework we call SSDLite. Additionally, we demonstrate how to build mobile semantic segmentation models through a reduced form of DeepLabv3 which we call Mobile DeepLabv3.

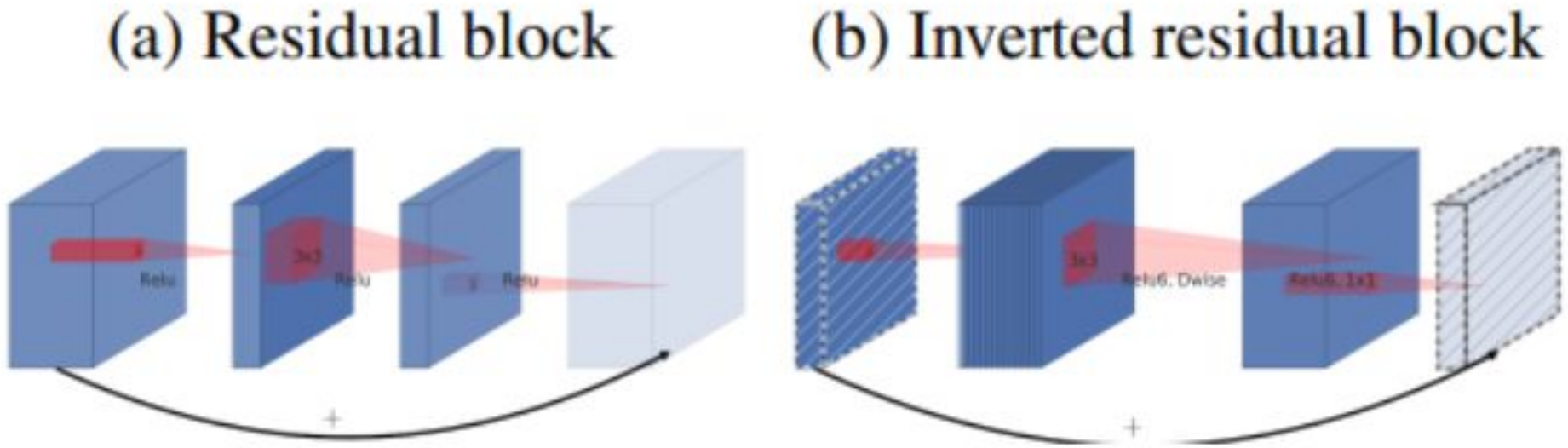
is based on an inverted residual structure where the shortcut connections are between the thin bottleneck layers. The intermediate expansion layer uses applications.

This paper introduces a new neural network architecture that is specifically tailored for mobile and resource constrained environments. Our network pushes the state of the art for mobile tailored computer vision models, by significantly decreasing the number of operations and memory needed while retaining the same accuracy.

Our main contribution is a novel layer module: the inverted residual with linear bottleneck. This module takes as an input a low-dimensional compressed representation which is first expanded to high dimension and filtered with a lightweight depthwise convolution. Features are subsequently projected back to a low-dimensional representation with a *linear convolution*. The official implementation is available as part of

Sandler, M. et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.

MobileNetV2: residual invertido



- A lógica é invertida: a entrada e a saída possuem **poucos canais**.
  - Primeiro ocorre uma **expansão** do número de canais.
  - O processamento principal ocorre nessa representação expandida.
  - Depois, os canais são novamente reduzidos por uma projeção.
- A conexão residual liga as representações **estreitas**, ou *bottlenecks*
  - blocos estreitos usam função de ativação linear para reduzir a complexidade (bottleneck)

MobileNetV2 reforça a tendência: blocos eficientes + atalhos para preservar informação.

Accuracy vs. efficiency

Experimentos executados na CIFAR-10

	Measurement Parameters			
Architecture	*Parameters*	*FLOPS*	*Memory*	*Accuracy*
AlexNet	58,322,314	~723 MFLOPS	223MB	82
Lenet-5	60000	~14.17 MFLOPS	234KB	76.23
SqueezeNet	742,258	~266 MFLOPS	2.83MB	92.5
ZFNet	58,302,986	~2327 MFLOPS	207MB	75.3
VGG16	134,301,514	~15000 MFLOPS	512MB	93.42
ResNet-50	23,608,202	~3870 MFLOPS	90.05MB	92.04
InceptionV1	5,985,082	~2000 MFLOPS	23MB	86.91
InceptionV3	21,823,274	~6000 MFLOPS	83.24MB	94.25
InceptionV4	33,432,778	~12000 MFLOPS	108 MB	98
Inception-ResNets V1	54,352,106	~11000 MFLOPS	207.3MB	80.1
Xception	20,881,970	~8986 MFLOPS	79.65MB	79
DenseNet-121	7,047,754	~2800 MFLOPS	26.88MB	91.86
Squeeze and Excitation	35,859,274	~12000 MFLOPS	136MB	95.6
MobileNetV1	3,239,114	~572 MFLOPS	12.35MB	77.8
NASNet	4,280,286	~564 MFLOPS	16.32MB	82.7
MobileNetV2	2,270,794	~300 MFLOPS	8.66MB	79.4
ShuffleNet	4,028,990	~524 MFLOPS	15.36MB	77.3
EffNet	3,246,874	~390 MFLOPS	12.38MB	85.2

N. Darapaneni, et. al. Convolution Neural Networks: A comparative Study for Image Classification. IEEE ICIIS. 2020

Qual a melhor?

… depende

Network	Accuracy (%)	Specificity (%)	Precision (%)	F1 score (%)
VGG16	57.72	78.89	58.11	52.63
GoogleNet	66.17	90.16	67.41	65.18
VGG19	69.32	91.41	69.01	68.43
InceptionV3	72.86	93.02	71.53	71.30
ResNet101	77.72	94.76	79.81	77.82

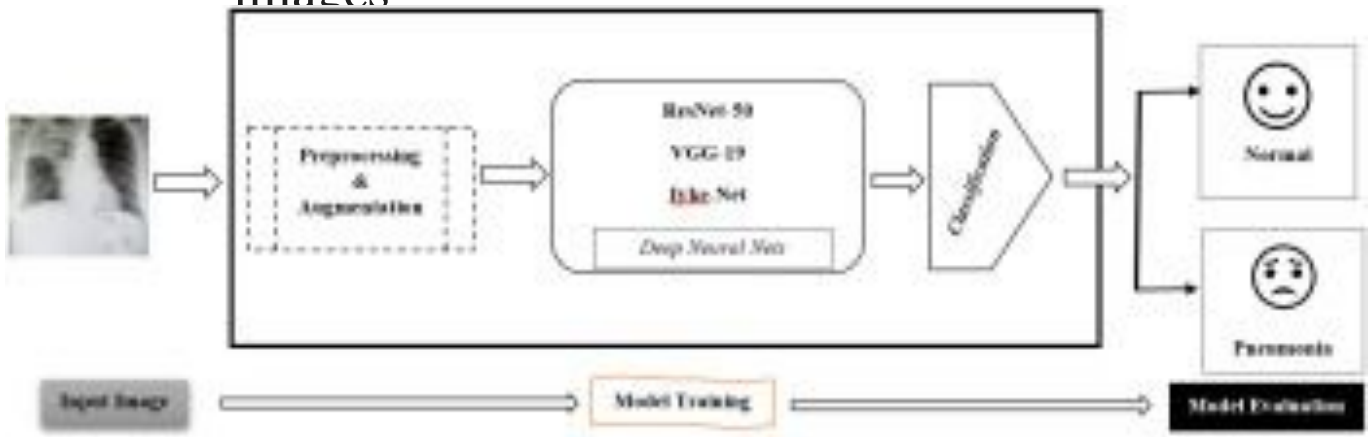
COMPARATIVE EVALUATIONS OF CNN BASED NETWORKS FOR SKIN LESION CLASSIFICATION

Method	Models	Accuracy (%)
Rajpukar etal(2017) [31]	ChexNet	85
Loey etal(2020) [32]	ResNet-50	84.5
Apostolopoulos etal(2020) [33]	VGG-19	93.5
Proposed method	Proposed (Iyke-Net)	**93.60**

ResNet-50 vs VGG-19 vs training from scratch: A comparative analysis of the segmentation and classification of Pneumonia from chest X-ray images

Technique	Method/ Algorithm	Epochs	Accuracy	Training Time
Build from scratch	CNN	50	97.5%	>1 hour
Transfer learning	VGG16	10	99.7%	<40 min.
	InceptionV3	10	99.7%	<1 hour
	MobileNet	10	99.9%	<30 min.
	ResNet	10	99.9%	<30 min.

DCNN-Based Vegetable Image Classification Using Transfer Learning: A Comparative Study



Usando um modelo (código)

ImageNet classification with Python and Keras

```python
# import the necessary packages
from keras.preprocessing import image as image_utils
from imagenet_utils import decode_predictions
from imagenet_utils import preprocess_input
from vgg16 import VGG16
import numpy as np
import argparse
import cv2

# construct the argument parse and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-i", "--image", required=True,
    help="path to the input image")
args = vars(ap.parse_args())

# load the original image via OpenCV so we can draw on it and display
# it to our screen later
orig = cv2.imread(args["image"])
```

Exemplo obtido de:
http://www.pyimagesearch.com/2016/08/10/imagenet-classification-with-python-and-keras/

Usando um modelo (código)

ImageNet classification with Python and Keras

```python
# load the input image using the Keras helper utility while ensuring
# that the image is resized to 224x224 pxiels, the required input
# dimensions for the network -- then convert the PIL image to a
# NumPy array
print("[INFO] loading and preprocessing image...")
image = image_utils.load_img(args["image"], target_size=(224, 224))
image = image_utils.img_to_array(image)
```

Usando um modelo (código)

VGG16, VGG19 ou ResNet50

ImageNet classification with Python and Keras

```python
# load the VGG16 network
print("[INFO] loading network...")
model = VGG16(weights="imagenet")

# classify the image
print("[INFO] classifying image...")
preds = model.predict(image)
(inID, label) = decode_predictions(preds)[0]

# display the predictions to our screen
print("ImageNet ID: {}, Label: {}".format(inID, label))
cv2.putText(orig, "Label: {}".format(label), (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)
cv2.imshow("Classification", orig)
cv2.waitKey(0)
```

$ python test_imagenet.py --image images/dog_beagle.png

Tunagem fina de um modelo

```
# example of tending the vgg16 model
from keras.applications.vgg16 import VGG16
from keras.models import Model
from keras.layers import Dense
from keras.layers import Flatten
# load model without classifier layers
model = VGG16(include_top=False, input_shape=(300, 300, 3))
# add new classifier layers
flat1 = Flatten()(model.layers[-1].output)
class1 = Dense(1024, activation='relu')(flat1)
output = Dense(10, activation='softmax')(class1)
# define new model
model = Model(inputs=model.inputs, outputs=output)
# summarize
model.summary()
# ...
```

https://machinelearningmastery.com/how-to-use-transfer-learning-when-developing-convolutional-neural-network-models/

Tunagem fina de um modelo

```python
# load model without classifier layers
model = VGG16(include_top=False, input_shape=(300, 300, 3))
# mark some layers as not trainable
model.get_layer('block1_conv1').trainable = False
model.get_layer('block1_conv2').trainable = False
model.get_layer('block2_conv1').trainable = False
model.get_layer('block2_conv2').trainable = False
...
```

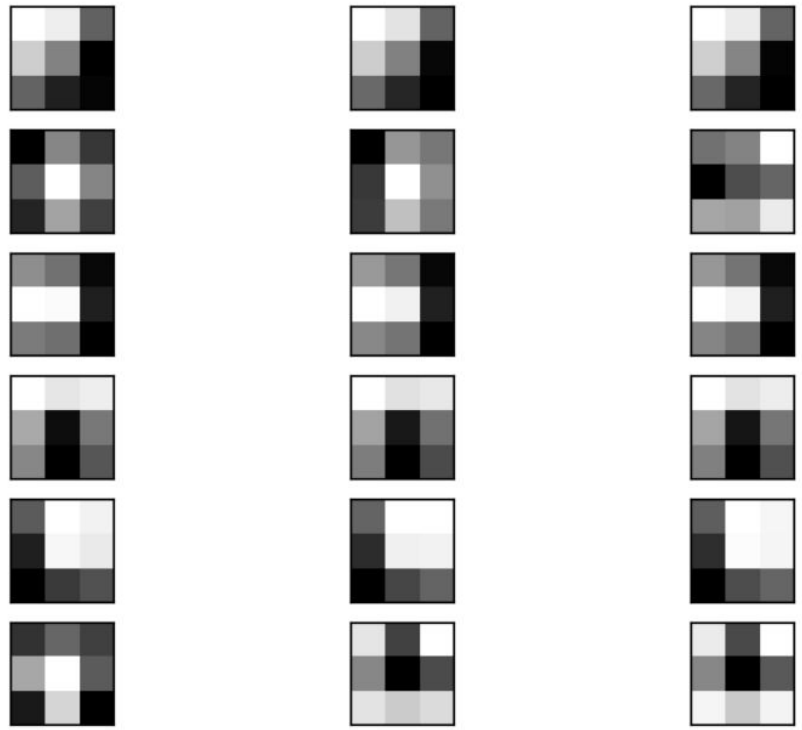
```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         (None, 300, 300, 3)       0
_________________________________________________________________
block1_conv1 (Conv2D)        (None, 300, 300, 64)      1792
_________________________________________________________________
block1_conv2 (Conv2D)        (None, 300, 300, 64)      36928
_________________________________________________________________
block1_pool (MaxPooling2D)   (None, 150, 150, 64)      0
_________________________________________________________________
block2_conv1 (Conv2D)        (None, 150, 150, 128)     73856
_________________________________________________________________
block2_conv2 (Conv2D)        (None, 150, 150, 128)     147584
_________________________________________________________________
block2_pool (MaxPooling2D)   (None, 75, 75, 128)       0
_________________________________________________________________
block3_conv1 (Conv2D)        (None, 75, 75, 256)       295168
_________________________________________________________________
block3_conv2 (Conv2D)        (None, 75, 75, 256)       590080
_________________________________________________________________
block3_conv3 (Conv2D)        (None, 75, 75, 256)       590080
_________________________________________________________________
block3_pool (MaxPooling2D)   (None, 37, 37, 256)       0
_________________________________________________________________
block4_conv1 (Conv2D)        (None, 37, 37, 512)       1180160
_________________________________________________________________
block4_conv2 (Conv2D)        (None, 37, 37, 512)       2359808
_________________________________________________________________
block4_conv3 (Conv2D)        (None, 37, 37, 512)       2359808
_________________________________________________________________
block4_pool (MaxPooling2D)   (None, 18, 18, 512)       0
_________________________________________________________________
block5_conv1 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block5_conv2 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block5_conv3 (Conv2D)        (None, 18, 18, 512)       2359808
_________________________________________________________________
block5_pool (MaxPooling2D)   (None, 9, 9, 512)         0
_________________________________________________________________
flatten_1 (Flatten)          (None, 41472)             0
_________________________________________________________________
dense_1 (Dense)              (None, 1024)              42468352
_________________________________________________________________
dense_2 (Dense)              (None, 10)                10250
=================================================================
Total params: 57,193,290
Trainable params: 57,193,290
Non-trainable params: 0
_________________________________________________________________
```

https://machinelearningmastery.com/how-to-use-transfer-learning-when-developing-convolutional-neural-network-models/

Interpretando as Features de uma arquitetura CNN

```python
# cannot easily visualize filters lower down
from keras.applications.vgg16 import VGG16
from matplotlib import pyplot
# load the model
model = VGG16()
# retrieve weights from the second hidden layer
filters, biases = model.layers[1].get_weights()
# normalize filter values to 0-1 so we can visualize them
f_min, f_max = filters.min(), filters.max()
filters = (filters - f_min) / (f_max - f_min)
# plot first few filters
n_filters, ix = 6, 1
for i in range(n_filters):
 # get the filter
 f = filters[:, :, :, i]
 # plot each channel separately
 for j in range(3):
 # specify subplot and turn of axis
 ax = pyplot.subplot(n_filters, 3, ix)
 ax.set_xticks([])
 ax.set_yticks([])
 # plot filter channel in grayscale
 pyplot.imshow(f[:, :, j], cmap='gray')
 ix += 1
# show the figure
pyplot.show()
```

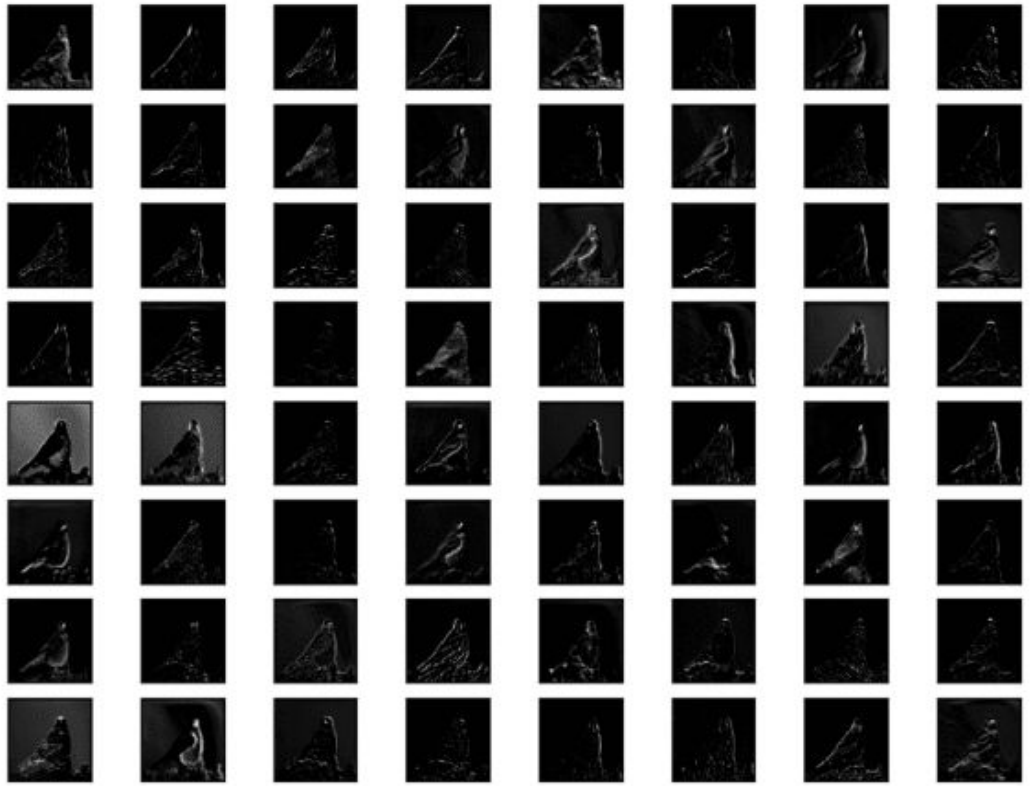
Filtros da camada [1]:



https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/

Interpretando as Features de uma arquitetura CNN

```python
# plot feature map of first conv layer for given image
from keras.applications.vgg16 import VGG16
from keras.applications.vgg16 import preprocess_input
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.models import Model
from matplotlib import pyplot
from numpy import expand_dims
# load the model
model = VGG16()
# redefine model to output right after the first hidden layer
model = Model(inputs=model.inputs, outputs=model.layers[1].output)
model.summary()
# load the image with the required shape
img = load_img('bird.jpg', target_size=(224, 224))
# convert the image to an array
img = img_to_array(img)
# expand dimensions so that it represents a single 'sample'
img = expand_dims(img, axis=0)
# prepare the image (e.g. scale pixel values for the vgg)
img = preprocess_input(img)
# get feature map for first hidden layer
feature_maps = model.predict(img)
# plot all 64 maps in an 8x8 squares
square = 8
ix = 1
for _ in range(square):
 for _ in range(square):
 # specify subplot and turn of axis
 ax = pyplot.subplot(square, square, ix)
 ax.set_xticks([])
 ax.set_yticks([])
 # plot filter channel in grayscale
 pyplot.imshow(feature_maps[0, :, :, ix-1], cmap='gray')
 ix += 1
# show the figure
pyplot.show()
```

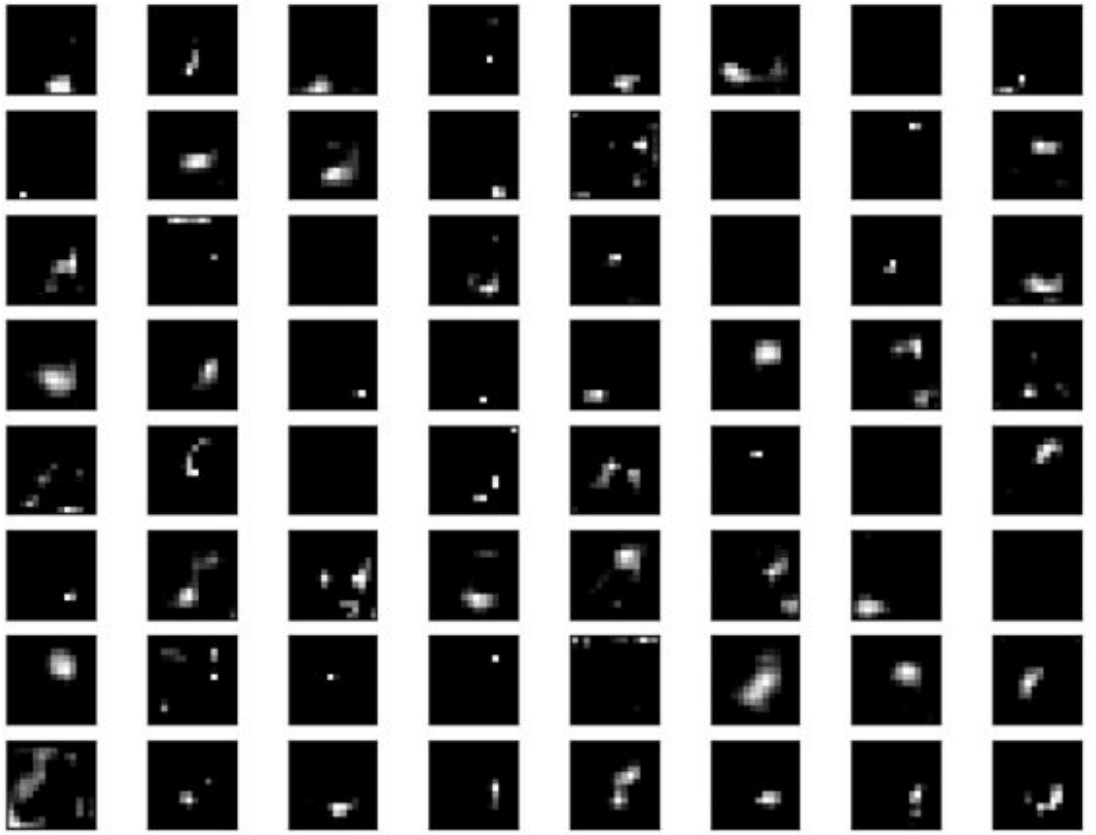




https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/

Interpretando as Features de uma arquitetura CNN

```python
# plot feature map of first conv layer for given image
from keras.applications.vgg16 import VGG16
from keras.applications.vgg16 import preprocess_input
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.models import Model
from matplotlib import pyplot
from numpy import expand_dims
# load the model
model = VGG16()
# redefine model to output right after the first hidden layer
model = Model(inputs=model.inputs, outputs=model.layers[1].output)
model.summary()
# load the image with the required shape
img = load_img('bird.jpg', target_size=(224, 224))
# convert the image to an array
img = img_to_array(img)
# expand dimensions so that it represents a single 'sample'
img = expand_dims(img, axis=0)
# prepare the image (e.g. scale pixel values for the vgg)
img = preprocess_input(img)
# get feature map for first hidden layer
feature_maps = model.predict(img)
# plot all 64 maps in an 8x8 squares
square = 8
ix = 1
for _ in range(square):
 for _ in range(square):
 # specify subplot and turn of axis
 ax = pyplot.subplot(square, square, ix)
 ax.set_xticks([])
 ax.set_yticks([])
 # plot filter channel in grayscale
 pyplot.imshow(feature_maps[0, :, :, ix-1], cmap='gray')
 ix += 1
# show the figure
pyplot.show()
```





https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/

Carregando uma imagem arbitrária

Após importar o modelo VGG do keras, é necessário carregar a imagem com o módulo image da biblioteca keras.preprocessing

```
from keras.applications.vgg16 import VGG16
model = VGG16(weights='imagenet')
print(model.summary())
```

```
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import
preprocess_input,decode_predictions
import numpy as np

img_path = '/kaggle/input/images/dog.jpg'
#There is an interpolation method to match the source size with the
target size
#image loaded in PIL (Python Imaging Library)
img = image.load_img(img_path,color_mode='rgb', target_size=(224,
224))
display(img)
```

Carregando uma imagem arbitrária

Por fim, é necessário converter a imagem para o tamanho compatível com o VGG16 e fazer a predição ou treinamento.

```
# Converts a PIL Image to 3D Numy Array
x = image.img_to_array(img)
x.shape
# Adding the fouth dimension, for number of images
x = np.expand_dims(x, axis=0)
```

```
#mean centering with respect to Image
x = preprocess_input(x)
features = model.predict(x)
p = decode_predictions(features)
```

https://towardsdatascience.com/how-to-use-a-pre-trained-model-vgg-for-image-classification-8dd7c4a4a517

Tratamento de Imagens de Alta Resolução (VGG16/VGG19)

Como preparar entradas no formato esperado sem distorcer o conteúdo original

Passo 1 & 2

Redimensionamento e Crop

Imagens de entrada típicas são maiores que os **224×224** exigidos pela VGG. Evite redimensionar diretamente ou cortar de forma arbitrária para evitar distorções.

1. Resize Proporcional:

Ajusta o menor lado para ~256px.
Ex: **1920×1080 → 455×256**

2. Center Crop:

Recorta o centro para a entrada ideal.
Ex: **341×256 → 224×224**

O comando abaixo distorce a imagem:

```
img = load_img("imagem.jpg", target_size=(224, 224))
```

Não distorce mas não centraliza:

```
img = load_img("imagem.jpg", target_size=(224, 224), keep_aspect_ratio=True)
```

Passo 3

Normalização e Pré-processamento Específico

Após o ajuste espacial, aplique estritamente o pré-processamento esperado pela implementação da VGG no TensorFlow/Keras:

```
from tensorflow.keras.applications.vgg16 import preprocess_input
x = preprocess_input(x)
```

Atenção com o Padrão de Normalização

A VGG do Keras utiliza o esquema clássico do ImageNet: converte a imagem de**RGB para BGR** e subtrai as médias dos canais

NÃO FAÇA: Não normalize dividindo diretamente por 255 (x / 255.0).

REGRA: Ao usar preprocess_input, não aplique nenhuma outra normalização prévia.

Resize e Crop na Mão

Implementação manual do pipeline de ajuste de dimensão e pré-processamento VGG16/VGG19

ETAPAS DO PIPELINE

Fluxo de Processamento

Este código executa exatamente as quatro etapas necessárias para preparar imagens mantendo o aspecto original sem distorções:

1. Resize Proporcional

Redimensiona mantendo a proporção de forma que o menor lado fique com 256px.

2. Center Crop 224x224

Extrai a região central no tamanho exato exigido pela arquitetura VGG.

3. Conversão para Array

Transforma a imagem PIL recortada em estrutura NumPy 3D.

4. Normalização VGG

Aplica a subtração das médias do ImageNet e a conversão de RGB para BGR.

Python / TensorFlow / PIL (Python Image Library)

```python
from tensorflow.keras.utils import load_img, img_to_array
from tensorflow.keras.applications.vgg16 import preprocess_input
from PIL import Image
import numpy as np

img = load_img("imagem.jpg") #imagem PIL

# 1. Resize proporcional: menor lado = 256
w, h = img.size
if w < h:
new_w = 256
new_h = round(h * 256 / w)
else:
new_h = 256
new_w = round(w * 256 / h)
img = img.resize((new_w, new_h))

# 2. Center crop 224x224
w, h = img.size
left = (w - 224) // 2
top = (h - 224) // 2
img = img.crop((left, top, left + 224, top + 224))

# 3. Converter para array
x = img_to_array(img)

# 4. Pré-processamento da VGG
x = preprocess_input(x)
```

Pré-processamento para Outras Arquiteturas (ResNet, EfficientNet, MobileNet)

Cada família de modelos exige um pipeline de entrada e faixa de valores específicos

PADRÃO RESNET / INCEPTION

ResNet50 & InceptionV3

Entrada Esperada: 224x224 (ResNet) ou 299x299 (Inception).

Escalonamento: Converte valores para o intervalo **[-1, 1]** ou aplica subtracão de média ImageNet.

```
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
from tensorflow.keras.preprocessing import image
img = image.load_img(path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
```

PADRÃO EFFICIENTNET

EfficientNet (B0 a B7)

Entrada Esperada: Variável conforme o modelo (ex: B0=224x224, B7=600x600).

Normalização Incluída: A própria camada do modelo realiza o scaling para **[0, 255]** internamente.

```
from tensorflow.keras.applications.efficientnet import EfficientNetB0, preprocess_input
img = image.load_img(path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
# Preprocess é neutro/pass-through no Keras recente
x = preprocess_input(x)
```

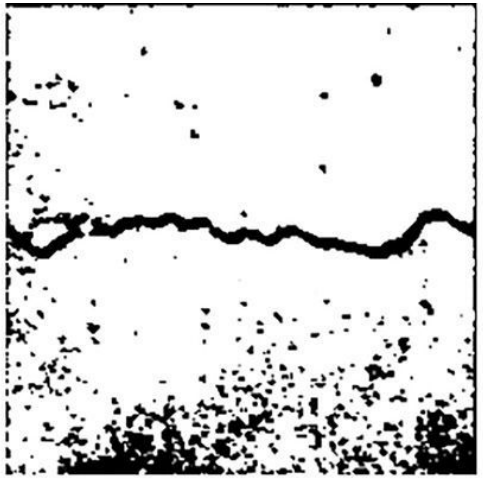
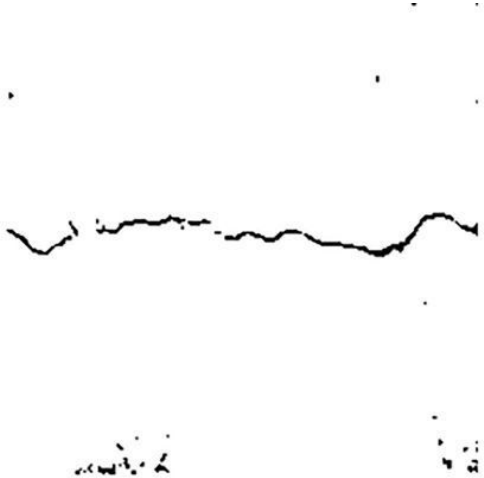
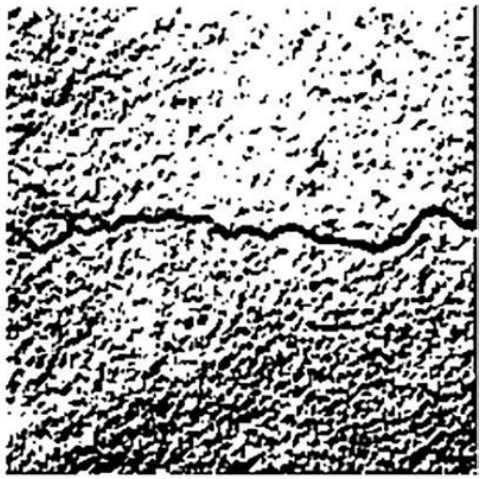
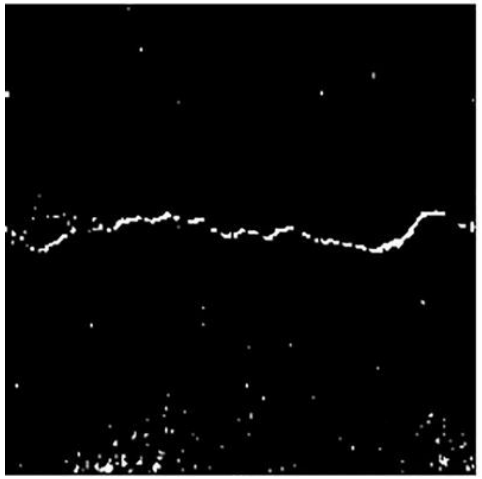
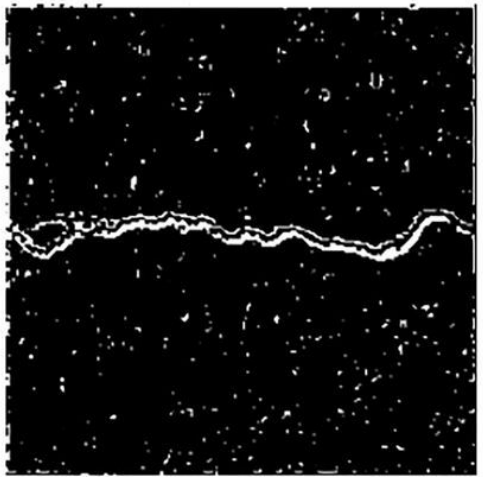
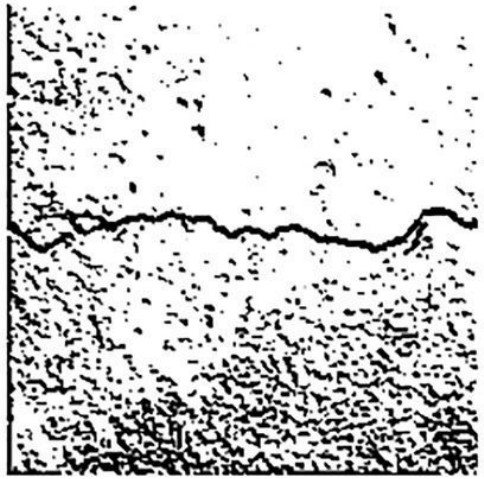
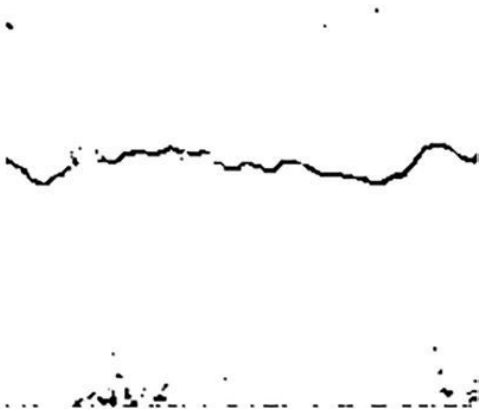
Cuidados

1. Sempre use o preprocess_input específico:
Cada sub-módulo de `tf.keras.applications` possui a sua própria função otimizada.

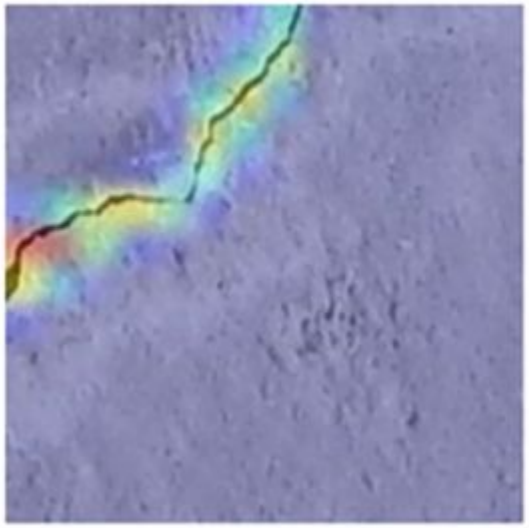
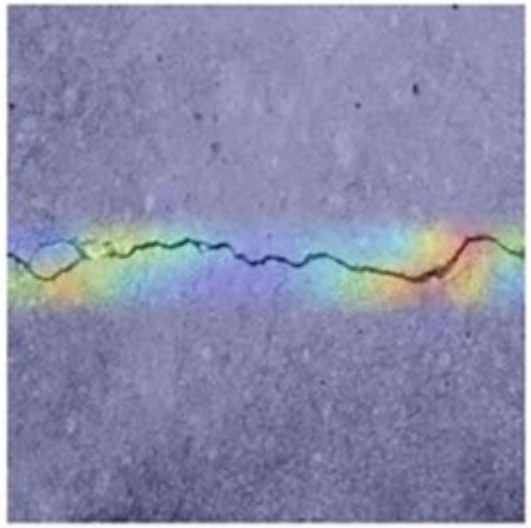
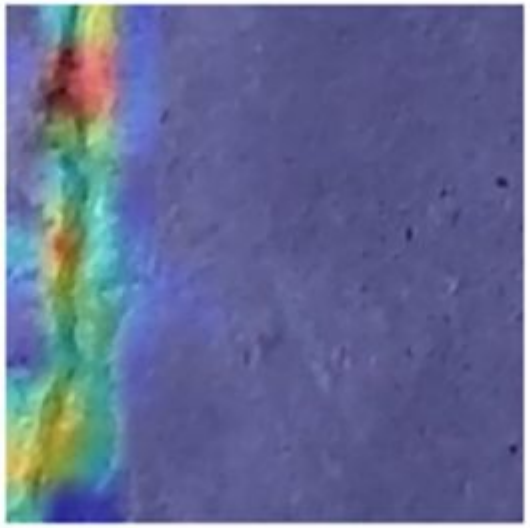
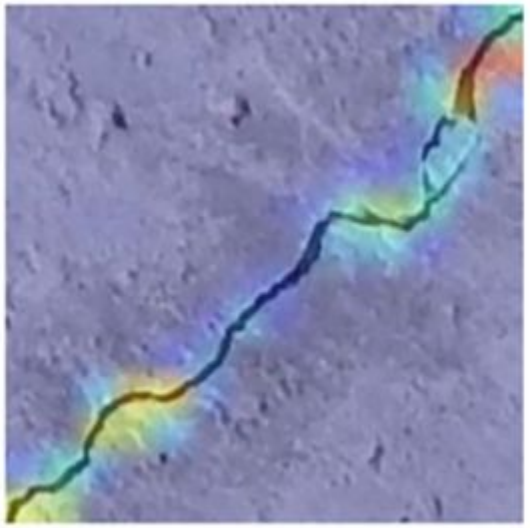
2. Respeite o target_size:
Redimensionar no `load_img()` garante ajuste automático sem estourar a memória GPU.

3. Batch Dimension:
O Keras exige o formato 4D `(batch, height, width, channels)`. Use sempre `np.expand_dims(x, axis=0)` antes de predizer.

Como identificar a região da imagem responsável pela classificação?

Como identificar a região da imagem responsável pela classificação?

Como identificar a região da imagem responsável pela classificação?

Implementação (keras):

- https://keras.io/examples/vision/grad_cam/